Architectural/Engineering “Punch List” Documentation

Software Requirement Specifications (SRS)

Version 2.0

9/16/2026

Nathan Rubiano, Muhammad Khan, Satya Thakur



Table of Contents

Software Requirements Specification (SRS).....1
Table of Contents.....2
1. Introduction.....5
1.1 - Purpose.....5
1.2 - Intended Audience.....5
1.3 - Scope.....5
1.4 - Definition.....6
1.5 - References.....8
2. Overall Description.....8
2.1 - Description.....8
2.2 - User Classes.....9
2.3 - Operating Environment.....10
2.4 - Constraints.....10
2.5 - Assumptions.....11
2.6 - Dependencies.....12
3. Business Rules.....14
3.1 Lifecycle & Role Authority.....16
3.2 Subcontractor Trade Boundary Isolation.....16
3.3 Resolution Verification Evidence.....17
3.4 Spatial Coordinate Normalization.....17
4. Functional Requirements.....18
4.1 - Authentication.....18
4.2 - Role-Based Authorization.....18
4.3 - Contractor/Subcontractor Relationships.....19
4.4 - Project Membership & Access Control.....20
4.5 - Project Management.....20
4.6 - Offline & Local Caching Architecture.....21
4.7 - Blueprint Sheets & Plan Management.....22
4.8 - Punch Item & Pin Lifecycle.....22
4.9 - Markups, Annotations & Photos.....23
4.10 - Executive Reporting & Document Export.....23
4.11 - Real-Time Collaboration & WebSockets.....24
4.12 - Audit Trail & System Logging.....24
5. Use Cases.....25
UC-01: User Login.....25
UC-02: Create a Project.....26

UC-03: Upload an Architectural/Engineering PDF……27
UC-04: Drop a Punch Pin (Creating a Task).....28
UC-05: Attach a Photo to a Punch Item……29
UC-06: Update Punch Item Status……31
UC-07: Assign a Punch Item to a User……32
UC-08: Add a Comment to a Punch Item……33
UC-09: Annotate a Blueprint with Markups……34
UC-10: Download an Annotated PDF Report……35
UC-11: Subcontractor Updates Assigned Work……37
UC-12: Offline Data Capture……38
UC-13: Manage Team Members……39
6. Data Requirements……41
6.1 - User……41
6.2 - Company……41
6.3 - ProjectCompany……41
6.4 - Project……41
6.5 - ProjectMembership……41
6.6 - RefreshToken……42
6.7 - ChangeLog……42
6.8 - ClientOperation……42
6.9 - Sheet……42
6.10 - SheetVersion……43
6.11 - Pin……43
6.12 - PinAssignee……43
6.13 - PinPhoto……43
6.14 - Comment……44
6.15 - Markup……44
6.16 - AuditLog……44
7. External Interface Requirements……45
8. Nonfunctional Requirements……50
8.1 - Security……50
8.2 - Performance……50
8.3 - Reliability……51
8.4 - Maintainability……52
8.5 - Usability & Tablet Ergonomics……52
8.6 - Portability & Interoperability……53
8.7 - Auditability……53

9. Reporting Requirements and Supplemental Information……..54
9.1 - Report 1……54
9.1.1 - System must support one-click generation of:.....54
9.1.2 - Reports must be scoped to the active project……54
9.1.3 - All generated files openable by any PDF or spreadsheet application……54
9.1.4 - Dashboard displays real-time KPI cards……54
10. Glossary of Terms……...54
10.1 - Punch List……54
10.2 - Punch Walkthrough……54
10.3 - Punch Item……...54
10.4 - Markup……..54
10.5 - Blueprint Viewer……..54
10.6 - Status Lifecycle……...54
10.7 - RBAC - Role-Based Access Control……55
10.8 - Offline Sync Queue……55
10.9 - Revision Cloud……55
10.10 - JSON Web Token……55
10.11 - Audit Log……55
10.12 - PlanGrid……55
11. Appendix……55
11.1 - Frontend Framework……..55
11.2 - Styling……55
11.3 - Blueprint Rendering……55
11.4 - Offline Storage……55
11.5 - Backend……55
11.6 - Database ……..55
11.7 - Real-Time……..55
11.8 - Auth……55
11.9 - Reporting……..55
12. Approvals and Sign-off……..56

1. Introduction

1.1 - Purpose

- This document defines the software requirements for “PunchList Pro”, a tablet-first construction inspection and documentation platform. The system is designed to address a well-known gap in the construction industry: the near-total absence of effective, purpose-built software for managing architectural and engineering punch list walkthroughs. This SRS outlines the functional and non-functional requirements, user roles, system constraints, and design expectations to guide the development team and serve as a formal agreement between developers and stakeholders throughout the project lifecycle.

1.2 - Intended Audience

- This document is intended for the development team (Nathan Rubiano, Muhammad Khan, Satya Thakur), project stakeholders, and any future contributors or evaluators of the system.

1.3 - Scope

- PunchList Pro is a tablet-first, role-based web application designed to streamline construction punch-list management for architectural, engineering, and construction (AEC) project teams.

  The platform enables project teams to upload architectural and engineering PDF drawings, conduct interactive punch-list walkthroughs directly on digital documents, place location-specific defect pins, attach photographs, assign corrective actions, track remediation progress, and generate exportable, annotated PDF reports compatible with any standard PDF reader.

  The application implements a six-role Role-Based Access Control (RBAC) architecture, ensuring that every user has permissions appropriate to their responsibilities, project assignments, and operational scope.

  Role-Based Access Control (RBAC)

Role	Display Label	Scope & Core Capabilities
`admin`	Facility Administrator	Full system governance, project CRUD, user and role management, architectural sheet management, trade allocation oversight, and sole authority to close or delete punch-list items.
`general_contractor`	General Contractor	Manages trade allocations, assigns punch-list items, coordinates subcontractors, reviews remediation submissions, and approves or rejects completed fixes.
`sub_contractor`	Trade Subcontractor	Access restricted to assigned items. Updates status (Open → In Progress → Contractor Review), uploads photographic evidence, and submits completed work for review.
`architect`	Architect	Performs specification auditing, identifies and logs defects, creates vector-based drawing annotations, reviews architectural documentation, and generates formal reports.
`owner`	Owner / Client Representative	Read-only executive dashboard, real-time project walkthrough visibility, progress monitoring, and exportable formal PDF reports.
`field_inspector`	Field Inspector	Conducts on-site QA/QC walkthroughs, identifies deficiencies, drops location-specific defect pins, documents findings, and verifies code compliance.

1.4 - Definition

- Punch List – A construction industry document that identifies outstanding defects, incomplete work, deficiencies, and quality assurance/quality control (QA/QC) issues that must be addressed and resolved before a construction project can be formally completed and handed over.
- Punch Item / Pin – An individual, location-specific task created on a digital blueprint to document a construction deficiency, incomplete work, or inspection finding. Each punch item is represented by an interactive pin and may contain a description, assigned trade, responsible user, status, photographs, comments, and supporting documentation.

- Blueprint / Sheet – A digital architectural, engineering, or construction drawing uploaded to the system in PDF, PNG, or JPG format. Blueprints serve as the visual reference for identifying, locating, documenting, and tracking construction deficiencies throughout a project.
- Markup – A vector-based visual annotation created directly on a blueprint to highlight, identify, or communicate a specific area of concern. Supported annotations may include revision clouds, arrows, rectangles, circles, lines, and freehand drawings. Markups remain associated with their respective drawing sheets.
- Status Lifecycle – The controlled sequence of workflow states through which a punch item progresses from initial identification to final resolution. The enforced lifecycle is:
- **`Open → In Progress → Contractor Review → Admin Review → Closed`**
- Each transition is governed by role-based permissions. Subcontractors submit completed work for contractor review, general contractors approve or reject remediation, and administrators perform the final review and formally close the item.
- RBAC (Role-Based Access Control) – A security model that regulates access to application features, data, and operations according to a user's assigned role. PunchList Pro implements six distinct roles: Administrator, General Contractor, Subcontractor, Architect, Owner, and Field Inspector. Each role has explicitly defined permissions and access boundaries enforced at the backend API level.
- Remediation – The corrective work performed to resolve a documented punch item. Remediation includes updating task progress, documenting corrective actions, attaching photographic evidence, and submitting the completed work for review.
- Contractor Review – A workflow stage in which the general contractor evaluates a subcontractor's submitted corrective work, verifies supporting evidence, and either approves the remediation for administrative review or rejects it for further correction.
- Admin Review – The final verification stage in which the facility administrator evaluates contractor-approved remediation and determines whether the punch item meets the requirements for formal closure.
- PDF Report / Export – A downloadable, annotated document containing punch-list information, blueprint references, pin locations, descriptions, photographs, assignments, comments, and workflow statuses. Exported reports are designed to be independently viewable in standard PDF readers without requiring access to PunchList Pro.

1.5 References

- Autodesk PlanGrid — reference platform for construction field management

2. Overall Description

2.1 - Description

Updated Description

PunchList Pro is a full-stack, tablet-first construction management web application inspired by enterprise platforms such as PlanGrid and Procore. It provides architectural, engineering, and construction (AEC) teams with a centralized workspace for managing punch lists, conducting on-site inspections, annotating digital blueprints, tracking deficiency remediation, and generating field reports.

The frontend is developed using React 18 and Vite, featuring a responsive, tablet-first design system with modern glassmorphic UI components optimized for field use across tablets, desktops, and mobile devices. The backend is powered by Python FastAPI, with Supabase (PostgreSQL) serving as the primary database and centralized data persistence layer.

The platform incorporates offline-first capabilities through Dexie.js and IndexedDB, allowing users to continue working with locally stored data when network connectivity is unavailable. A background synchronization queue reconciles locally generated changes with Supabase once connectivity is restored.

PunchList Pro implements role-based authentication and authorization (RBAC) across six distinct user roles: Facility Administrator, General Contractor, Trade Subcontractor, Architect, Owner, and Field Inspector. Each role has clearly defined permissions governing project access, punch-item management, task assignments, remediation reviews, and administrative operations.

The application integrates interactive PDF blueprint viewing, location-specific punch-item pins, vector-based markups, photographic evidence, real-time workflow updates, and exportable annotated PDF reports.

The deployment architecture uses Vercel for frontend hosting, Render for backend API hosting, and Supabase for managed PostgreSQL database services and persistent application data.

The overall goal of PunchList Pro is to digitize construction punch-list workflows, improve coordination between project stakeholders, maintain accountability throughout the remediation lifecycle, and streamline project closeout through a unified, accessible, and reliable platform.

2.2 - User Classes

User Class	Display Label	Descriptions and Characteristics
`admin`	Facility Administrator	Has full system governance privileges, including project creation, retrieval, updating, and deletion (CRUD), user and role management, architectural sheet management, and trade allocation oversight. Holds exclusive authority to formally close or delete punch-list items.
`general_contractor`	General Contractor	Manages trade allocations, assigns punch-list items, coordinates subcontractors, monitors remediation progress, reviews submitted corrective work, and approves or rejects completed fixes before administrative review. Cannot independently perform final punch-item closure.
`sub_contractor`	Trade Subcontractor	Has access strictly limited to punch items assigned to their company or individual account. Can update item statuses (`Open` → `In Progress` → `Contractor Review`), upload photographic evidence, add comments, and submit completed work for contractor review. Cannot approve or close punch items.

`architect`	Architect	Performs specification auditing, identifies and documents construction deficiencies, creates and updates punch items, generates vector-based drawing annotations, reviews architectural documentation and completion evidence, and generates or exports formal project reports.
`owner`	Owner / Client Representative	Has read-only access to project information through an executive dashboard. Can monitor project progress, review punch-list items, follow real-time walkthrough updates, inspect remediation status, and export formal PDF reports. Cannot modify project data or approve, close, or delete punch items.
`field_inspector`	Field Inspector	Conducts on-site quality assurance and quality control (QA/QC) walkthroughs, identifies construction deficiencies, creates location-specific defect pins on blueprints, documents inspection findings, attaches supporting photographs, and verifies compliance with applicable construction specifications and codes.

2.3 - Operating Environment

- The system operates as a web-based application accessible via modern browsers (Chrome, Edge, Safari, Firefox) on desktop, laptop, tablet, and mobile devices. The frontend runs on port 3000 (Vite dev server) and the backend API runs on port 4000 (FastAPI + Uvicorn ASGI). The system is designed primarily for tablet use on active construction sites, including outdoor environments with direct sunlight. It must function in both fully connected and intermittently offline (low-connectivity jobsite) conditions. Supabase(PostgreSQL) is the primary production database; SQLite is used as an automatic fallback for local development environments.

2.4 - Constraints

- The frontend must function offline using Dexie.js (IndexedDB) and queue sync operations for when connectivity is restored.
- Blueprint rendering must support 4K-level zoom clarity without pixelation, using PDF.js and the HTML5 Canvas API.

- The backend must run on Python 3.14 with the pg8000 pure-Python driver (zero C-dependencies) for portability.
- Authentication must use JWT tokens with bcrypt password hashing — no third-party auth providers.
- The UI must meet tablet ergonomic standards: touch targets no smaller than 38px, no reliance on hover-only interactions.
- All status transitions for punch items must be validated server-side via the validate_status_transition service — the frontend cannot bypass lifecycle rules.

2.5 - Assumptions

- Field personnel using PunchList Pro are expected to operate primary tablet hardware running current Chrome, Safari, or Edge browsers. Desktop and mobile environments remain supported for administrative functions.
- Standard jobsite office network connectivity is assumed, though intermittent offline conditions in the field are handled via Dexie.js local storage and background synchronization.
- Uploaded architectural and engineering sheets are presumed to be compliant PDF, PNG, or JPG files conforming to established file size constraints.
- Active projects require minimum staffing consisting of one General Contractor (`general_contractor`) for trade coordination and one Facility Administrator (`admin`) for overall governance and item closure.
- User accounts must be explicitly provisioned by an Administrator prior to system access, with public self-registration disabled across all role tiers.
- System principals operate strictly within backend-enforced RBAC capabilities governing project membership, task allocation, and status transitions.
- Persistent data storage relies on Supabase PostgreSQL, with local client operations queuing automatically for server reconciliation upon network restoration.
- Exported PDF punch reports and annotated plan sets are readable using standard third-party PDF tools without active system accounts.
- Project team members are responsible for providing precise pin coordinates, accurate descriptions, supporting photos, and timely remediation updates.
- Production deployment assumes frontend hosting on Vercel, API hosting on Render, and database services powered by managed Supabase infrastructure.

2.6 - Dependencies

Dependency	Purpose
React 18	Provides the component-based frontend architecture for building interactive application interfaces and managing user interactions.
Vite	Handles frontend development, module bundling, and production build generation.
Python 3	Serves as the primary backend programming language for application logic and API development.
FastAPI	Provides the backend REST API, request validation, authentication integration, and business logic execution.
Supabase (PostgreSQL)	Provides centralized relational database storage for users, roles, projects, blueprint metadata, punch items, assignments, and application records.
Dexie.js / IndexedDB	Enables offline-first functionality by storing application data locally in the browser and supporting queued synchronization when connectivity is restored.
Vercel	Hosts and deploys the React frontend application, providing production hosting and frontend delivery.
Render	Hosts the Python FastAPI backend and provides the publicly accessible API endpoints consumed by the frontend.
PDF.js	Enables browser-based rendering, navigation, and viewing of uploaded architectural and engineering PDF drawings.

Dependency	Purpose
PDF Generation Libraries	Support the generation and export of annotated punch-list reports and construction documentation in standard PDF format.
JavaScript / CSS	Support frontend application behavior, responsive layouts, styling, and interactive tablet-first interfaces.
Git / GitHub	Provide source-code version control, collaborative development, branch management, and project change tracking.
Modern Web Browsers	Provide the runtime environment for accessing PunchList Pro on supported desktop, tablet, and mobile devices.
Internet Connectivity	Enables communication between the frontend, backend API, Supabase database services, and other integrated services.

External Service Dependencies

The application's production functionality depends on the availability and proper configuration of the following hosted services:

- Vercel: Frontend deployment, hosting, and environment configuration.
- Render: Backend API hosting, application execution, and environment configuration.
- Supabase: Managed PostgreSQL database, persistent data storage, and database connectivity.
- GitHub: Source-code repository and deployment integration.

3. Business Rules

Rule ID	Rule Definition	Type	Impacted Requirements
BR-01	Admin/Owner and Architect/Engineer may close punch items after verifying completion. Contractor and Subcontractor users may submit work for review but cannot close items. Only Admin/Owner may permanently delete an item, subject to retention and archive rules.	Authorization	`FR-RBAC-05`, `FR-PIN-08`
BR-02	A Subcontractor may view and act on a punch item only if it is assigned to their company or to them individually.	Authorization	`FR-ORG-09`, `FR-RBAC-08`
BR-03	A Contractor may view and act on punch items assigned to any subcontractor company that reports to them on that project.	Authorization	`FR-ORG-08`, `FR-RBAC-06`
BR-04	A user with no active membership on a project is denied access to that project and to every resource (sheets, pins, annotations, comments, photos) it contains.	Authorization	`FR-MEM-02`, `FR-AUTH-04`
BR-05	A user cannot modify their own project membership or grant themselves a higher project role. Role elevation requires an Admin.	Authorization	`FR-MEM-05`, `FR-RBAC-03`
BR-06	A punch item cannot be assigned to a subcontractor company that is not attached to that item's project.	Data Integrity	`FR-ORG-07`, `FR-PIN-04`
BR-07	A punch item status transition must be re-validated against the lifecycle state machine at the time it is applied to the server, even if it was queued while offline.	Workflow	`FR-SYNC-08`, `FR-PIN-05`
BR-08	An archived project accepts no mutations to itself or to its sheets, pins, comments, or photos; it is strictly read-only.	Workflow	`FR-PRJ-05`, `FR-RBAC-09`
BR-09	A company cannot be permanently deleted while it is referenced by an active project attachment or punch item assignment; it must be deactivated instead.	Data Integrity	`FR-ORG-10`, `FR-DB-04`

Rule ID	Rule Definition	Type	Impacted Requirements
BR-10	A password is never stored or logged in plain text; only its bcrypt hash is persisted.	Security	`FR-AUTH-02`, `FR-SEC-01`
BR-11	Moving a punch item status from In Progress to Contractor Review requires at least one proof-of-completion photo and a resolution comment.	Workflow	`FR-PIN-06`, `FR-DOC-03`
BR-12	Pin spatial coordinates on any drawing sheet must be stored and calculated strictly as normalized percentages relative to native sheet dimensions, preserving position invariance across all display resolutions, zoom levels, and aspect ratios.	Data Integrity	`FR-PIN-02`, `FR-SHEET-04`
BR-13	When an Admin or Contractor rejects a punch item under review, the status transitions back to In Progress, and a mandatory rejection reason must be supplied and appended to the item's comment thread.	Workflow	`FR-PIN-05`, `FR-COM-02`
BR-14	When a newer revision of an architectural or engineering sheet is published, existing pins and vector annotations must automatically carry over to the new sheet version unless explicitly repositioned.	Data Integrity	`FR-SHEET-06`, `FR-PIN-09`
BR-15	Offline mutation conflicts are resolved using server-authoritative timestamps: if an item was updated online while an offline client edited it, the server rejects invalid state transitions and logs a conflict resolution notification.	Offline / Sync	`FR-SYNC-04`, `FR-SYNC-09`
BR-16	All state-changing operations (pin creation, status transitions, reassignment, photo uploads, sheet revisions, deletions) must create an immutable record in audit_logs containing user ID, project ID, previous state, new state, and ISO 8601 UTC timestamp.	Audit	`FR-AUDIT-01`, `FR-AUDIT-03`
BR-17	Deleting a punch item performs a soft delete (deleted_at timestamp flag); items are never physically removed from the persistence store until project retention policies expire.	Data Integrity	`FR-PIN-10`, `FR-DB-06`

Rule ID	Rule Definition	Type	Impacted Requirements
BR-18	Generated export artifacts (annotated plan PDF, executive punch list PDF, CSV export) must strictly redact items that the requesting user's role is not authorized to view.	Security / Privacy	`FR-EXP-03`, `FR-RBAC-10`
BR-19	A punch pin marked with Critical priority must emit an alert notification to the assigned trade contractor and project GC immediately upon creation or escalation.	Notification	`FR-NOTIF-02`, `FR-PIN-07`
BR-20	Uploaded blueprint files must be verified as valid architectural formats (PDF, PNG, JPG) and cannot exceed the configured maximum payload size (50 MB) before processing.	Validation	`FR-SHEET-01`, `FR-SEC-05`

3.1 Lifecycle & Role Authority (BR-01, BR-07, BR-11, BR-13)

- **Stakeholder Feedback:** Initial field evaluations indicated that subcontractor personnel inadvertently updated deficiency tasks to a closed status on mobile hardware, thereby bypassing required general contractor oversight.
- **Refinement:** The five-stage workflow progression ($Open \rightarrow InProgress \rightarrow ContractorReview \rightarrow AdminReview \rightarrow Closed$) was restricted against subcontractor modification. Trade subcontractors are strictly limited to transitioning items to Contractor Review.
- **Test Case:**
  - *Input:* A subcontractor issues PATCH /api/pins/{id}/status providing status: "Closed".
  - *Expected Result:* The API returns HTTP 403 Forbidden alongside error payload INSUFFICIENT_ROLE_PRIVILEGES.

3.2 Subcontractor Trade Boundary Isolation (BR-02, BR-03, BR-06)

- **Stakeholder Feedback:** Trade teams operating on shared jobsites noted operational ambiguity and data privacy concerns regarding visibility into punch items and rate data belonging to separate subcontractor entities.
- **Refinement:** Database queries apply tenant-level authorization filters: Trade Subcontractors retrieve only pins matching assigned_company_id == user.company_id or assigned_user_id == user.id.
- **Test Case:**

- *Input:* An electrical subcontractor user attempts GET /api/pins/{id} referencing a task assigned to a plumbing subcontractor firm.
- *Expected Result:* The API returns HTTP 404 Not Found (or 403 Forbidden), fully masking external trade records.

3.3 Resolution Verification Evidence (BR-11)

- **Stakeholder Feedback:** Project supervisors identified instances of subcontractors submitting items for verification without performing required remediation or attaching photographic evidence.
- **Refinement:** Backend validation enforces that at least one photo record (photo_count >= 1) and a mandatory resolution comment exist before transitioning state from In Progress to Contractor Review.
- **Test Case:**
  - *Input:* A user submits a PATCH request to transition status to Contractor Review without attached photos.
  - *Expected Result:* The API returns HTTP 422 Unprocessable Entity containing message "Proof of completion photo required".

3.4 Spatial Coordinate Normalization (BR-12)

- **Stakeholder Feedback:** Inspection personnel examining high-resolution architectural PDFs across field tablets and desktop displays observed spatial misalignment when pins relied on absolute pixel offsets.
- **Refinement:** Pin coordinates are persisted as normalized (x_{pct}, y_{pct}) percentage values, allowing the blueprint viewer to calculate pixel positioning dynamically using: $$\text{pixel}x = \text{canvasWidth} \times \left(\frac{x{\text{pct}}}{100}\right), \quad \text{pixel}y = \text{canvasHeight} \times \left(\frac{y{\text{pct}}}{100}\right)$$
- **Test Case:**
  a. *Input:* Save pin location at $(x: 25.5\%, y: 64.2\%)$, then render across display viewports 1920×1080 and 3840×2160.
  b. *Expected Result:* Calculated relative position maintains accuracy within the $< 0.01\%$ tolerance threshold on both viewports.

4. Functional Requirements

4.1 - Authentication

- FR-AUTH-01: System access accounts are provisioned exclusively by a Facility Administrator with an email address, user full name, assigned system role, and company affiliation; public self-registration is explicitly disabled.
- FR-AUTH-02: User credentials are persisted solely using bcrypt password hashes generated with a per-password salt. Plaintext passwords are strictly prevented from appearing in data stores, application logs, or API payloads (see BR-10).
- FR-AUTH-03: The authentication service validates login requests against stored credentials to issue a short-lived JWT access token alongside an opaque refresh token, both evaluated and generated by the application backend.
- FR-AUTH-04: Transmitted JWT access tokens encapsulate user identity claims, system roles, and associated company identifiers, configured to expire 60 minutes after generation.
- FR-AUTH-05: Refresh tokens remain valid for 30 days, are stored in hashed format within the database, and permit clients to obtain updated access tokens without presenting raw user credentials.
- FR-AUTH-06: Unsuccessful login attempts return a uniform error response and status code regardless of failure reason to prevent account enumeration attacks.
- FR-AUTH-07: Protected REST API routes enforce authentication by requiring a valid, active access token in request headers, excluding public login, token refresh, and system health endpoints.
- FR-AUTH-08: Explicit user logout requests instantly invalidate and revoke the presented refresh token on the server.

4.2 - Role-Based Authorization

- FR-RBAC-01: The system enforces RBAC tiers comprising Facility Administrator (admin), General Contractor (general_contractor), Trade Subcontractor (sub_contractor), Architect (architect), Owner (owner), and Field Inspector (field_inspector). Operational scope and capabilities are defined per role.
- FR-RBAC-02: User capabilities are structured as discrete permission strings (e.g., pin.close, project.update) mapped inside a centralized access control module rather than hardcoded checks distributed across service routes.

- FR-RBAC-03: Authorization checks are enforced server-side on every request; interface control hiding functions strictly for tablet usability and does not serve as a security boundary.
- FR-RBAC-04: API calls initiated by user principals lacking required operational capabilities immediately return HTTP 403 Forbidden responses.
- FR-RBAC-05: Administrative and design roles hold exclusive authority to execute final punch-item closure. Contractor and subcontractor personnel may advance tasks to review stages but cannot close or delete deficiency records (see BR-01).
- FR-RBAC-06: Executive oversight privileges granted to an Owner role remain restricted to assigned project scopes and do not grant system-wide administrative control over unassigned project instances.
- FR-RBAC-07: The backend exposes an active capability query endpoint allowing field client applications to render actions corresponding to the principal's evaluated permissions.

4.3 - Contractor/Subcontractor Relationships

- FR-ORG-01: Company records maintain organizational metadata including company category (general contractor, subcontractor, architecture, owner) and designated trade specializations for subcontractors.
- FR-ORG-02: Each provisioned user account maintains a foreign key link binding them to one parent company record.
- FR-ORG-03: Facility Administrators and General Contractors may attach external company entities to active projects with designated project roles (prime, sub, design, owner).
- FR-ORG-04: Project assignments establish contractual reporting structures via parent_company_id links, permitting subcontractors to report to distinct prime contractors across individual jobsite projects.
- FR-ORG-05: Punch items accept task assignments to a trade company, an individual user, or both, subject to an enforced maximum of two total assignee records. The backend validates and rejects operations exceeding this cap during online requests and offline sync operations.
- FR-ORG-06: Task assignment operations attempting to assign punch items to subcontractor entities not linked to the active project are rejected by server validation (see BR-06).
- FR-ORG-07: General Contractors retain full visibility and modification authority over punch-list items assigned to subordinate trade firms within their project structure (see BR-03).
- FR-ORG-08: Trade Subcontractors are strictly isolated to punch items assigned to their specific company or user account, preventing unauthorized inspection of external trade records (see BR-02).

- FR-ORG-09: Database foreign key constraints prevent the physical deletion of company records referenced by active project assignments or punch items, requiring soft deactivation instead (see BR-09).

4.4 - Project Membership & Access Control

- FR-MEM-01: Project level access is established through explicit ProjectMembership entities associating user profiles with specific project roles.
- FR-MEM-02: Principals without an active ProjectMembership record are completely restricted from accessing project data, blueprint sheets, deficiency pins, comments, or attachments (see BR-04).
- FR-MEM-03: User authorization scope is determined by the evaluated project role assigned within ProjectMembership; global administrative access requires explicit system administrator status.
- FR-MEM-04: Authorized Administrators and General Contractors manage project rosters by adding project members, modifying assigned roles, and deactivating access.
- FR-MEM-05: Server permissions prohibit project members from modifying their own membership records or elevating their own project role capabilities (see BR-05).
- FR-MEM-06: Project list endpoints return only projects where the requesting user maintains an active membership, reserving full system project queries for system administrators.
- FR-MEM-07: Unauthenticated attempts to access unauthorized project resources return HTTP 404 Not Found responses to prevent project existence enumeration.

4.5 - Project Management

- FR-PRJ-01: System principals with project creation capabilities can instantiate project records specifying project title, unique project number, site address, and owner entity.
- FR-PRJ-02: Project creation requires a valid name and unique project identifier; missing fields trigger validation errors and halt record insertion.
- FR-PRJ-03: Project metadata and operational attributes can be updated by authorized project administrators.
- FR-PRJ-04: Projects transition across defined lifecycle states: active, archived, and closed.

- FR-PRJ-05: Archived projects transition to a strictly read-only state, allowing data retrieval and report generation while rejecting all modification attempts to child sheets or pins (see BR-08).
- FR-PRJ-06: Projects containing linked plan sheets or punch list records are protected from physical database deletion; archiving serves as the official project completion mechanism.
- FR-PRJ-07: Successful project creation automatically provisions an active ProjectMembership entry assigning the creator as project administrator.
- FR-PRJ-08: The backend provides executive dashboard summary endpoints aggregating punch item metrics categorized by status lifecycle and priority.

4.6 - Offline & Local Caching Architecture

- FR-SYNC-01: Data mutation operations create incremental audit entries in the change_log table indexed with monotonically increasing sequence values.
- FR-SYNC-02: Delta synchronization endpoints process project identifiers and client sync sequence cursors to return pending mutation entries along with updated sequence markers.
- FR-SYNC-03: Server delta payloads are filtered by the principal's authorization boundaries, ensuring client IndexedDB (Dexie.js) stores retain only authorized project data.
- FR-SYNC-04: Resource deletions append tombstone markers to the delta stream, instructing offline tablet clients to remove corresponding local records upon synchronization.
- FR-SYNC-05: Batch sync endpoints receive queued offline client operations, identifying each payload item with a unique client-generated operation UUID.
- FR-SYNC-06: The server tracks processed operation UUIDs to enforce idempotency, preventing duplicate record creation when handling retried synchronization batches.
- FR-SYNC-07: Field entities generated offline utilize client-side UUID primary keys, preserving entity relationships during subsequent backend reconciliation.
- FR-SYNC-08: Offline status updates undergo server-side re-validation against validate_status_transition rules during batch processing (see BR-07).
- FR-SYNC-09: Batch responses return status details for each sync operation (applied, rejected, superseded); individual failures do not invalidate the entire transaction batch.
- FR-SYNC-10: Photographic evidence captured offline uploads via separate multipart HTTP requests linked to synchronized punch records, optimizing core metadata sync speed.
- FR-SYNC-11: Real-time updates propagate over Socket.IO channels scoped to active project rooms, refreshing client views without requiring manual page reloads.

- FR-SYNC-12: Hydration endpoints deliver complete project dataset snapshots (sheets, pins, comments, team assignments) for initial offline cache population.

4.7 - Blueprint Sheets & Plan Management

- FR-SHT-01: Authorized personnel can upload architectural and engineering drawings in PDF, PNG, or JPG formats up to 50 MB per file payload.
- FR-SHT-02: Drawing sheets require unique sheet identifiers (e.g., A-101, E-201), descriptive titles, trade discipline designations, and optional building level tags.
- FR-SHT-03: Vector PDF plans render via PDF.js and HTML5 Canvas API, maintaining crisp 4K-level detail up to 800% zoom scale on desktop and tablet displays.
- FR-SHT-04: Publishing updated drawing sheets creates a new version record while preserving historical plan revisions in the document store.
- FR-SHT-05: Existing punch pins and vector markups automatically migrate to new plan sheet revisions using normalized relative coordinates (see BR-14).
- FR-SHT-06: Sheet deletion is blocked while active punch items remain linked; pins must be relocated or resolved prior to sheet removal.

4.8 - Punch Item & Pin Lifecycle

- FR-PIN-01: Inspection users place location-specific defect pins on digital blueprints, capturing title details, trade classification, room designation, descriptions, and priority levels.
- FR-PIN-02: Spatial pin locations are stored exclusively as normalized percentage values ($x_{pct}, y_{pct} \in [0.00, 100.00]$) relative to native sheet dimensions, maintaining display accuracy across device aspect ratios (see BR-12).
- FR-PIN-03: Deficiency tasks progress through an enforced five-stage workflow: Open → In Progress → Contractor Review → Admin Review → Closed.
- FR-PIN-04: Advancing task status to Contractor Review requires submitting resolution comments and attaching proof-of-completion photographs (see BR-11).
- FR-PIN-05: Rejection of remediation work by Contractors or Administrators returns the task status to In Progress and appends mandatory rejection comments to the discussion thread (see BR-13).

- FR-PIN-06: Final item closure is strictly reserved for Facility Administrator and Architect roles following verification (see BR-01).
- FR-PIN-07: Punch item deletion applies a soft-delete timestamp flag (deleted_at); database records are retained for auditing while the parent project remains active (see BR-17).
- FR-PIN-08: Deficiency pins flagged with Critical priority highlight visually in the UI and generate immediate alert notifications to responsible trade contractors (see BR-19).

4.9 - Markups, Annotations & Photos

- FR-DOC-01: Users with design annotation permissions create vector markups (revision clouds, arrows, rectangles, freehand lines) on transparent overlay layers above drawings.
- FR-DOC-02: Drawing markups are persisted as vector geometry JSON objects bound to specific sheet versions and rendered dynamically in the blueprint viewer.
- FR-DOC-03: Field personnel attach inspection photographs to punch items using local file selection or direct tablet camera integration.
- FR-DOC-04: Uploaded photo files are stored on secure server storage and referenced via URL paths; binary image contents are excluded from relational database tables.
- FR-DOC-05: Photo upload records store metadata including uploader ID, generation timestamp, image orientation, and generated thumbnail URLs.
- FR-DOC-06: Project team members post time-stamped comments within punch item discussion threads to maintain communication records during remediation.

4.10 - Executive Reporting & Document Export

- FR-EXP-01: Authorized roles (Admin, General Contractor, Architect, Owner) generate exportable PDF punch reports detailing project summaries, item statuses, pin indices, and attached photos.
- FR-EXP-02: The report generation service produces annotated plan sets embedding vector markups and pin markers directly onto architectural drawing PDFs.
- FR-EXP-03: Exported document artifacts conform to standard PDF specifications, ensuring compatibility across desktop and mobile PDF viewers without requiring proprietary software.
- FR-EXP-04: The platform generates tabular CSV data exports containing punch item metadata, statuses, trade assignments, and coordinate positions for integration with external management software.

- FR-EXP-05: Exported files automatically redact deficiency records outside the requesting user's role authorization scope (see BR-18).

4.11 - Real-Time Collaboration & WebSockets

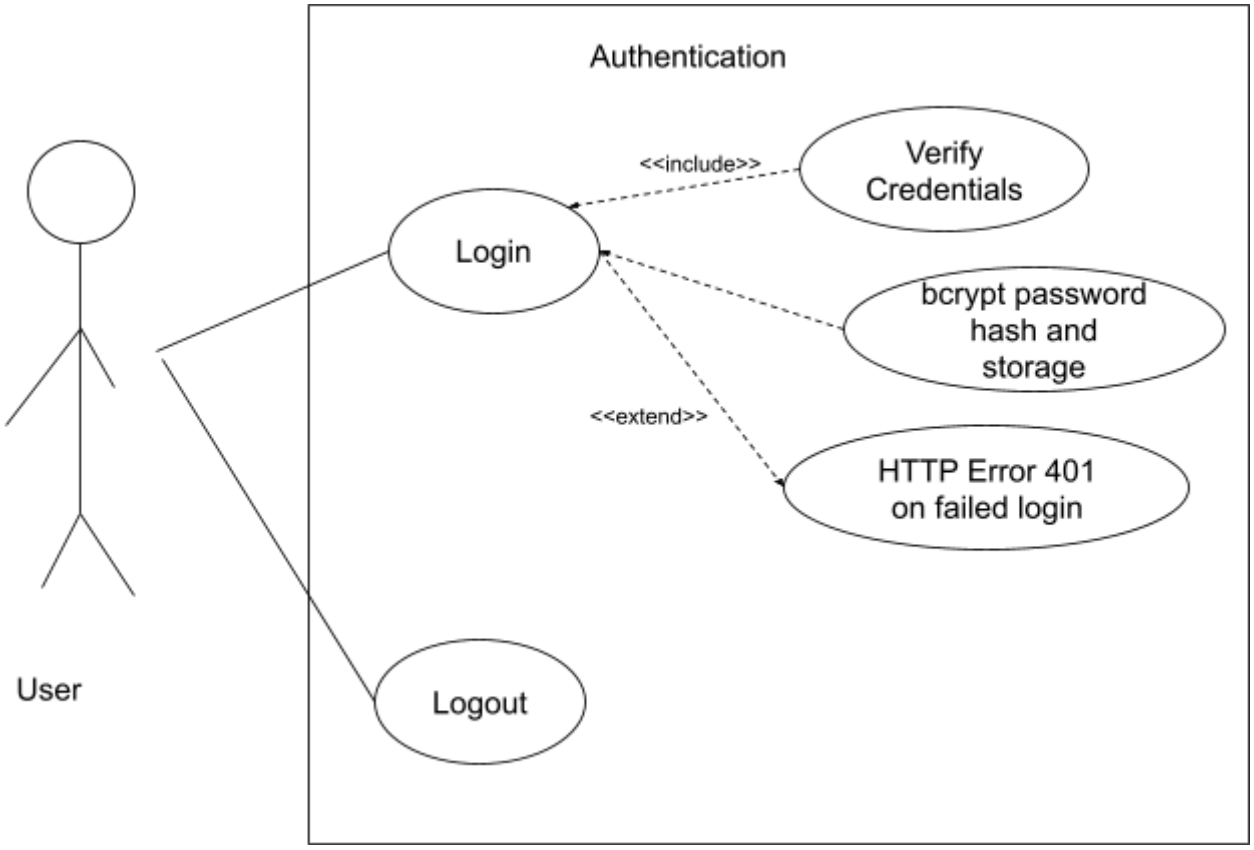
- FR-RT-01: The backend maintains bidirectional Socket.IO connections, categorizing connected clients into project-specific channels (project_{id}).
- FR-RT-02: Pin updates, position moves, and status transitions broadcast to active project room clients within 500 ms of server event processing.
- FR-RT-03: Blueprint pin overlays update dynamically upon receiving WebSocket messages without requiring full page refreshes or canvas re-renders.
- FR-RT-04: Network disconnections cause clients to fall back to HTTP polling, triggering sync queue reconciliation immediately upon restoring connectivity.

4.12 - Audit Trail & System Logging

- FR-AUD-01: Data modifications involving projects, sheets, punch items, comments, and project memberships generate append-only audit entries in audit_logs (see BR-16).
- FR-AUD-02: Audit records capture user IDs, target entities, operation types (CREATE, UPDATE, DELETE, STATUS_CHANGE), state snapshots, and ISO 8601 UTC timestamps.
- FR-AUD-03: System audit logs are permanently append-only and protected against modification or deletion by any user role.
- FR-AUD-04: Audit log REST endpoints expose activity streams filtered by project ID for display within administrative reporting dashboards.

5. Use Cases

UC-01: User Login



- FR-AUTH-01: The system shall provide a login endpoint accepting an email address and password, authenticating against pre-provisioned user records with no open self-registration.

- FR-AUTH-02: Passwords shall be validated against stored bcrypt hashes. Plaintext passwords shall never be logged, stored, or transmitted in response payloads (see BR-10).

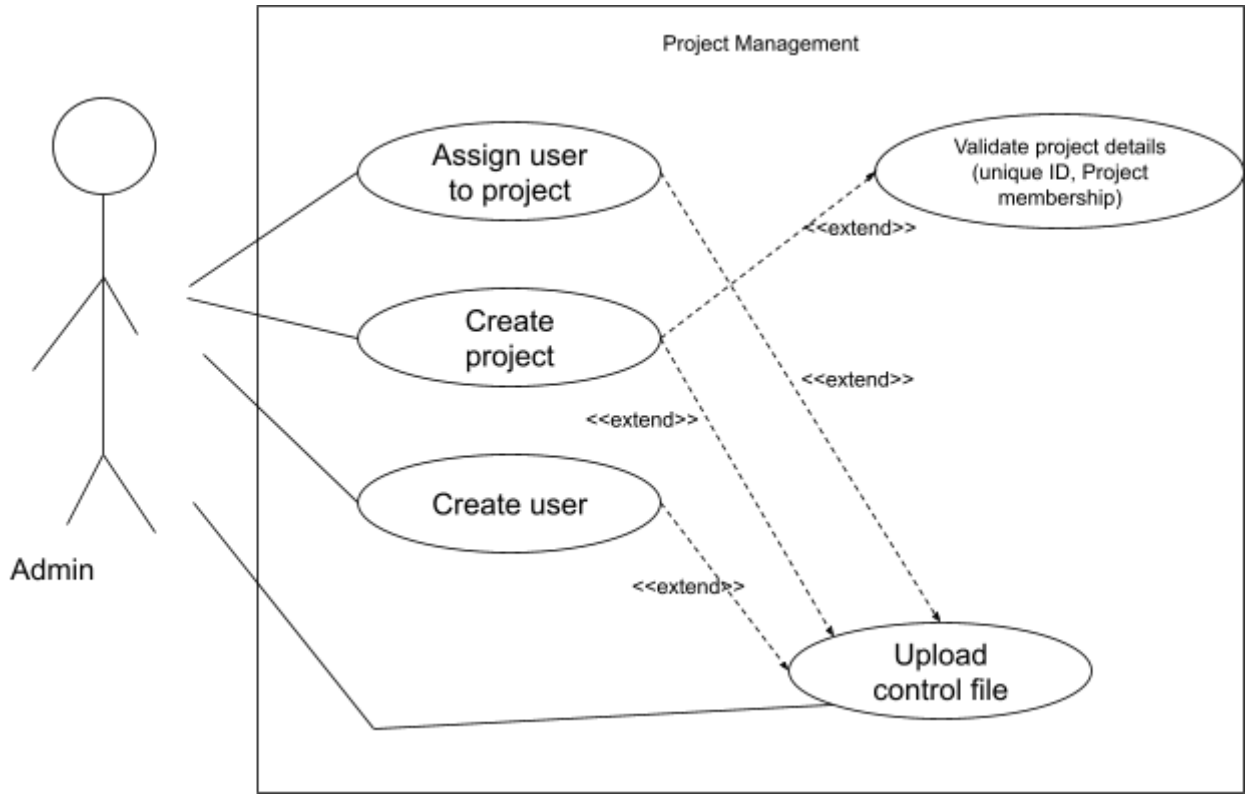
- FR-AUTH-03: Upon successful credential verification, the system shall return a cryptographically signed JWT access token and an opaque refresh token.

- FR-AUTH-04: The access token shall embed claims for user ID, system role, and affiliated company ID, with an expiration window of 60 minutes.

- FR-AUTH-05: Any failed authentication attempt with an invalid email or invalid password shall return a uniform HTTP 401 error message to prevent user enumeration.

- FR-AUTH-06: Upon successful authentication, the client shall store session tokens and route the user to their role-appropriate project dashboard.

UC-02: Create a Project



- FR-PRJ-01: A user authenticated with the Facility Administrator role can access the project creation interface.

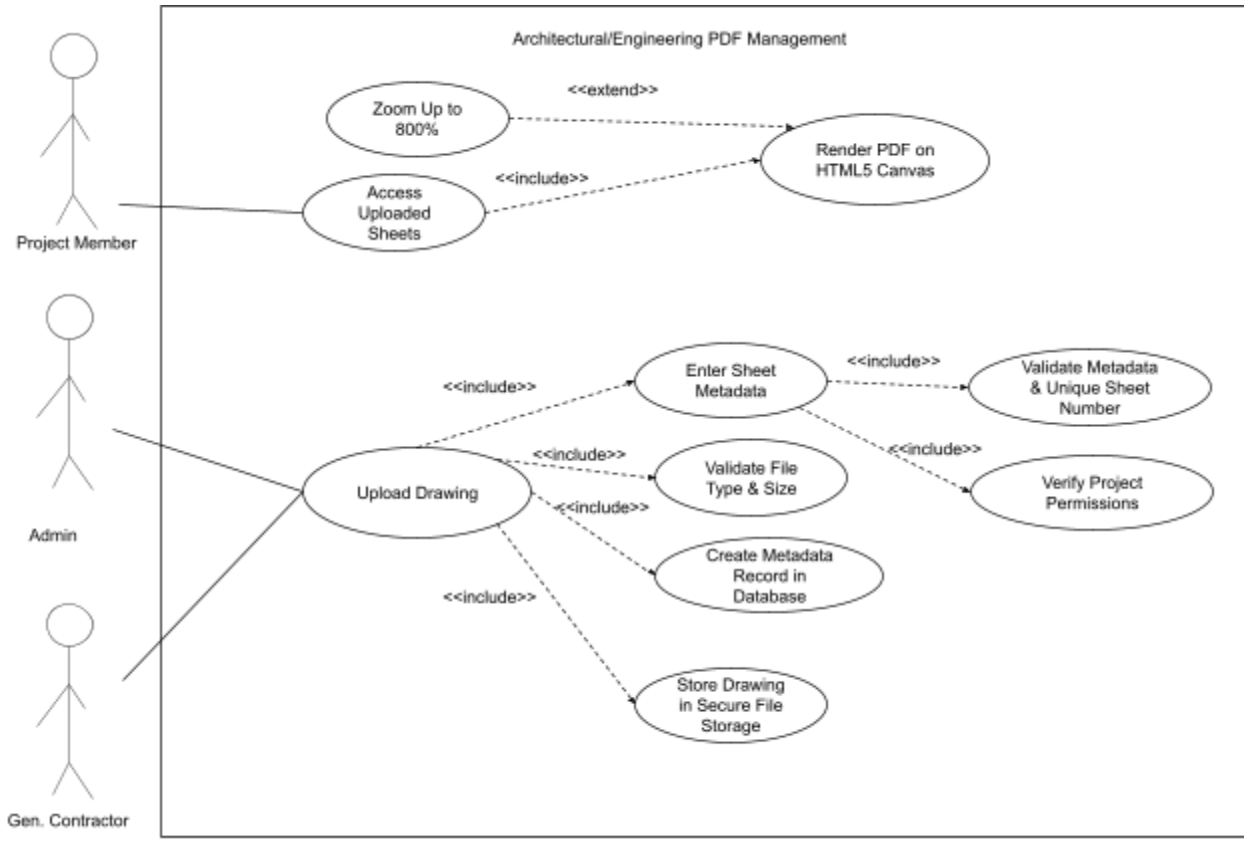
- FR-PRJ-02: The project creation endpoint shall require a non-empty project name, a unique project code/number, a physical site address, and a target closeout date.

- FR-PRJ-03: The system shall reject duplicate project numbers with an HTTP 422 field-level validation error.

- FR-PRJ-04: Creating a project shall automatically insert an active project membership record binding the creating administrator to that project (see BR-04).

- FR-PRJ-05: Upon creation, the new project record shall be immediately queryable in the project listing and available for sheet ingestion and team provisioning.

UC-03: Upload an Architectural/Engineering PDF



- FR-SHT-01: A user holding the Admin or General Contractor role with sheet management capability can upload drawing files in vector PDF, PNG, or JPG formats up to 50 MB.

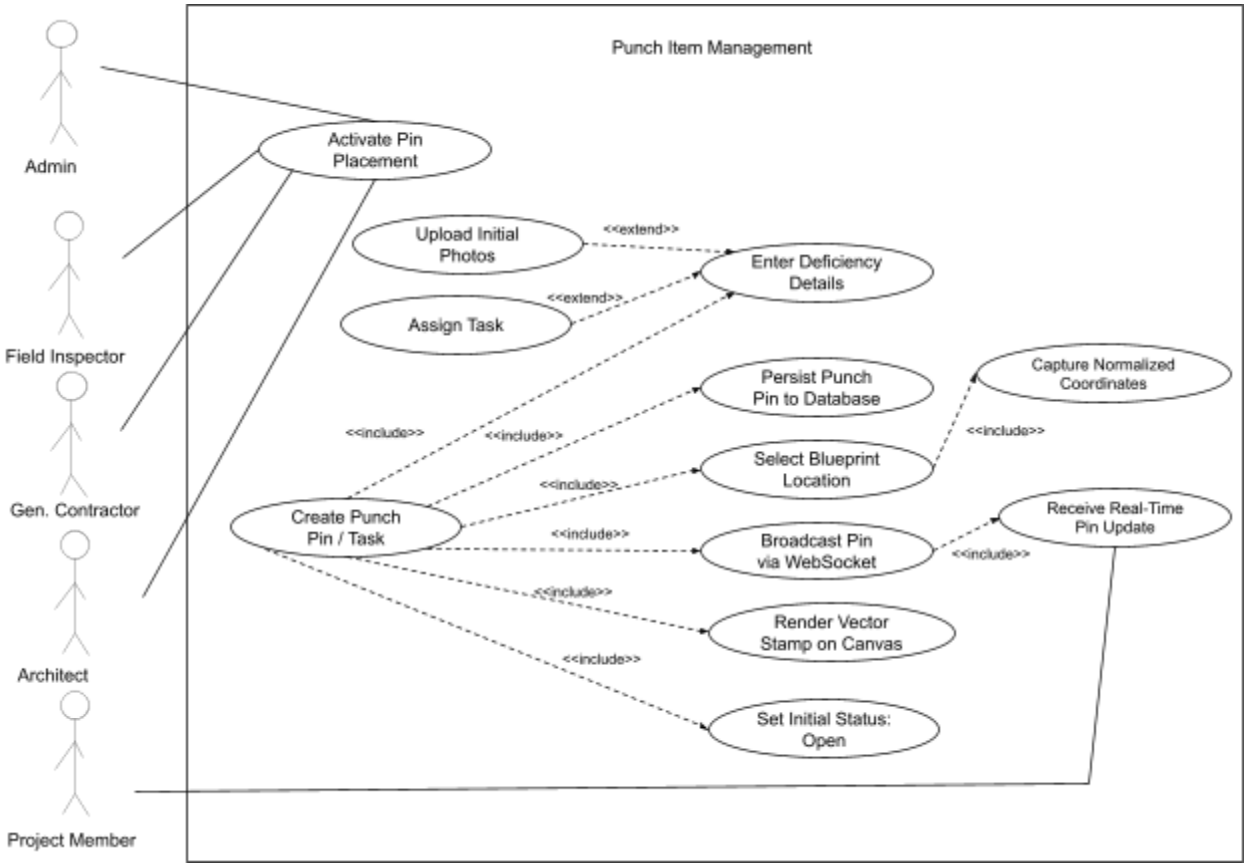
- FR-SHT-02: The upload form shall require metadata including building identifier, floor level, sheet title, unique sheet number (e.g., A-101), and discipline classification (Architectural, Structural, MEP, or Civil).

- FR-SHT-03: The server shall store the raw drawing asset in secure file storage and generate a persistent metadata record in the database.

- FR-SHT-04: The web application shall render vector PDF plan sheets onto an HTML5 canvas at high resolution, maintaining crisp linework and dimension text up to 800% zoom.

- FR-SHT-05: Uploaded sheets shall be immediately accessible to all active project members according to their project permissions.

UC-04: Drop a Punch Pin (Create a Task)



- FR-PIN-01: Users with pin creation capability (Admin, General Contractor, Architect, and Field Inspector) can activate the pin placement tool during a walkthrough.

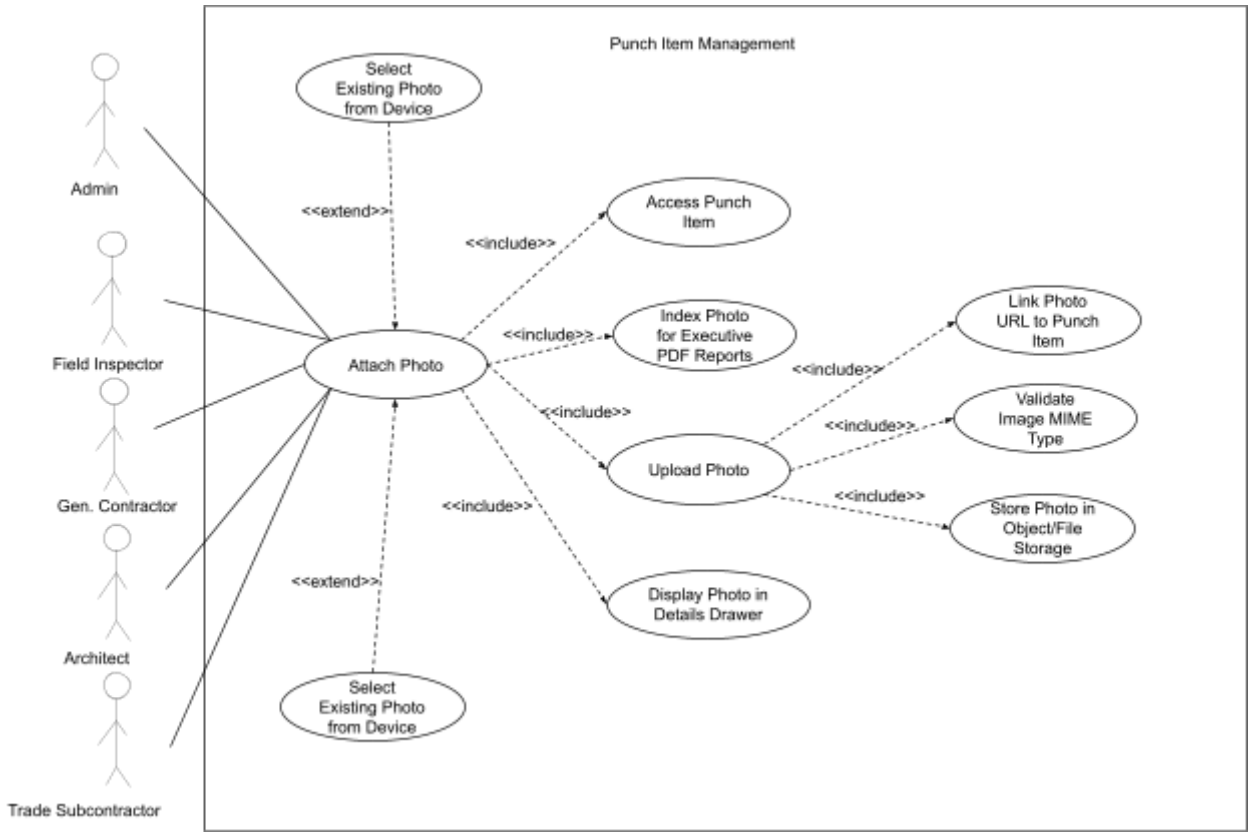
- FR-PIN-02: Selecting a point on the blueprint canvas shall capture the location strictly as normalized percentage coordinates ($x_{pct}, y_{pct} \in [0.00, 100.00]$) relative to the sheet dimensions.

- FR-PIN-03: The punch pin creation modal shall capture a deficiency description, trade/discipline category, priority level (Low, Medium, High, Critical), room/area location, initial status (Open), optional assignee, initial photos, and remarks.

- FR-PIN-04: The system shall render a vector stamp icon on the canvas at the coordinate point, displaying the trade initial and status color ring.

- FR-PIN-05: Newly created pins shall be persisted to the database and broadcast in real time to all connected clients in the project's WebSocket room within 500 ms.

UC-05: Attach a Photo to a Punch Item



- FR-MED-01: Users with access to a punch item (Admin, General Contractor, Architect, Field Inspector, or assigned Trade Subcontractor) can attach field photos.

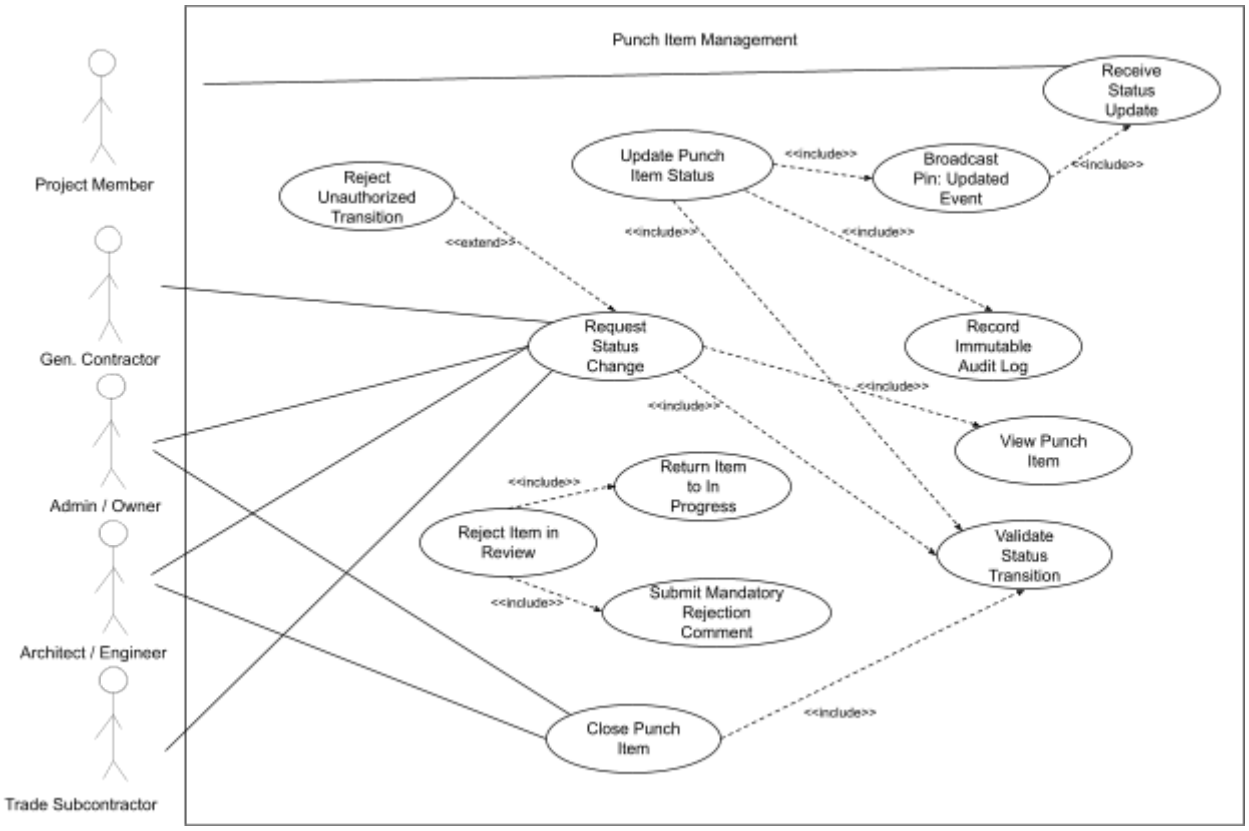
- FR-MED-02: The client interface shall support direct camera capture on tablet/mobile devices as well as file selection from device storage.

- FR-MED-03: The backend shall accept multi-part image uploads, validate image MIME types, store files in object/file storage, and record the returned URL against the punch item.

- FR-MED-04: Raw photo binary data shall not be stored directly in relational database rows.

- FR-MED-05: Linked photos shall render immediately in the punch item details drawer and be indexed for inclusion in executive PDF reports.

UC-06: Update Punch Item Status



- FR-STAT-01: The system shall enforce a strict 5-stage lifecycle state machine: Open → In Progress → Contractor Review → Admin Review → Closed.

- FR-STAT-02: The server-side service validate_status_transition shall evaluate all status change requests against the user's role and current item status before applying updates (see BR-07).

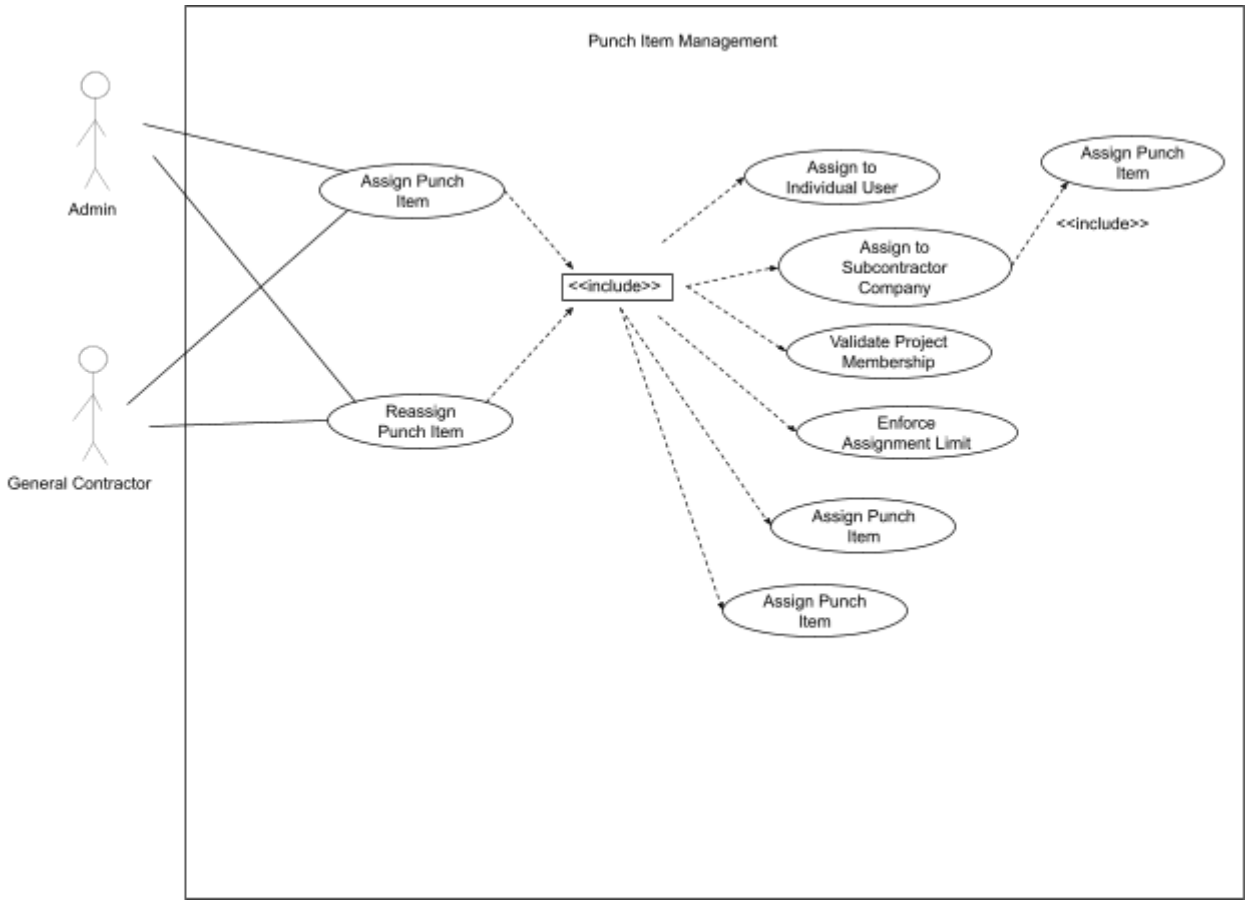
- FR-STAT-03: Admin/Owner and Architect/Engineer may transition an item to Closed; Contractor and Subcontractor requests to close an item shall be rejected with HTTP 403 (see BR-01).

- FR-STAT-04: Rejecting an item in review transitions its status back to In Progress and requires a mandatory rejection comment.

- FR-STAT-05: Every status update shall generate an immutable record in audit_logs detailing previous status, new status, actor ID, and timestamp.

- FR-STAT-06: Successful status updates shall emit a pin:updated event across the project's Socket.IO channel.

UC-07: Assign a Punch Item to a User



- FR-ASN-01: An Admin or General Contractor can assign or reassign a punch item to an individual project member, an attached subcontractor company, or both.

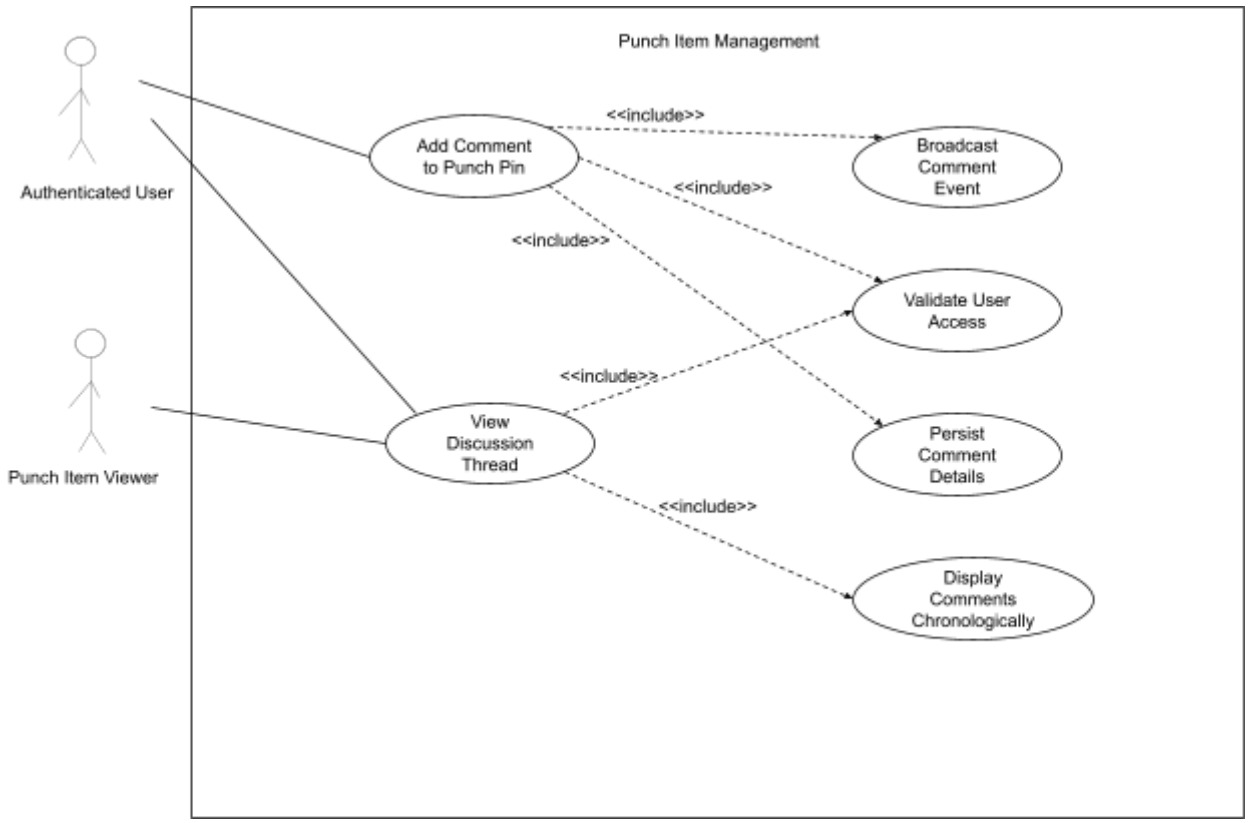
- FR-ASN-02: The system shall reject assigning an item to any subcontractor company not attached to the parent project (see BR-06).

- FR-ASN-03: Total assignee entries on a single punch item shall not exceed two (maximum one company and one individual user). Any request exceeding this cap shall return HTTP 422.

- FR-ASN-04: Assigning a subcontractor company grants visibility to all verified members of that company who hold active project membership, without adding each user individually.

- FR-ASN-05: Assigning an item updates its search indexes and surfaces the item in the assignee's personal dashboard filter.

UC-08: Add a Comment to a Punch Item



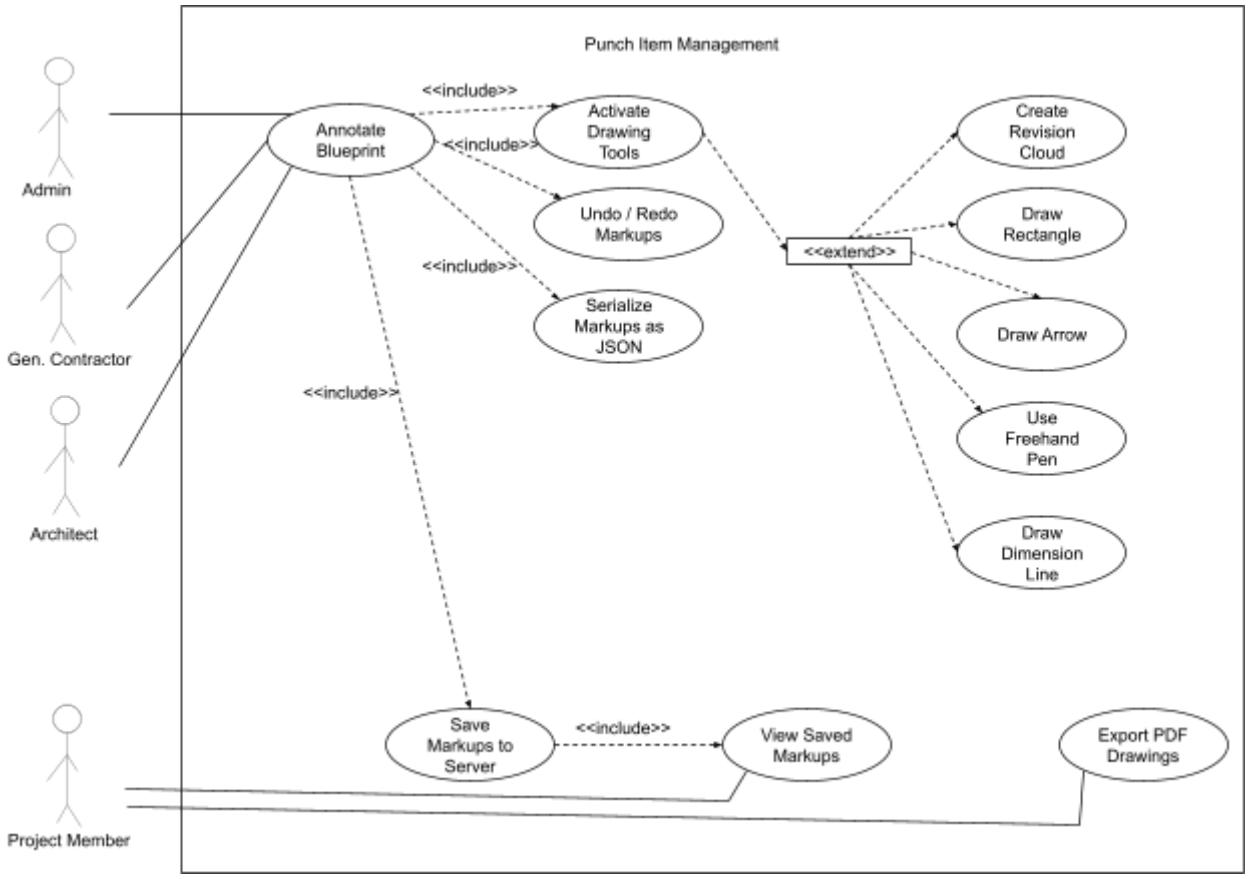
- FR-COM-01: Any authenticated user with authorized access to a punch item can submit textual comments to the item's discussion thread.

- FR-COM-02: Each comment record shall persist the author's user ID, display name, user role, text body, and microsecond UTC timestamp.

- FR-COM-03: Comment threads shall be rendered in strict chronological order and shall be immutable; comments cannot be edited or deleted by standard users.

- FR-COM-04: Posting a comment shall broadcast an event to active viewers of that punch item via WebSockets.

UC-09: Annotate a Blueprint with Markups



- FR-MKP-01: Users with markup capabilities (Admin, General Contractor, Architect) can activate drawing tools on the transparent overlay canvas.

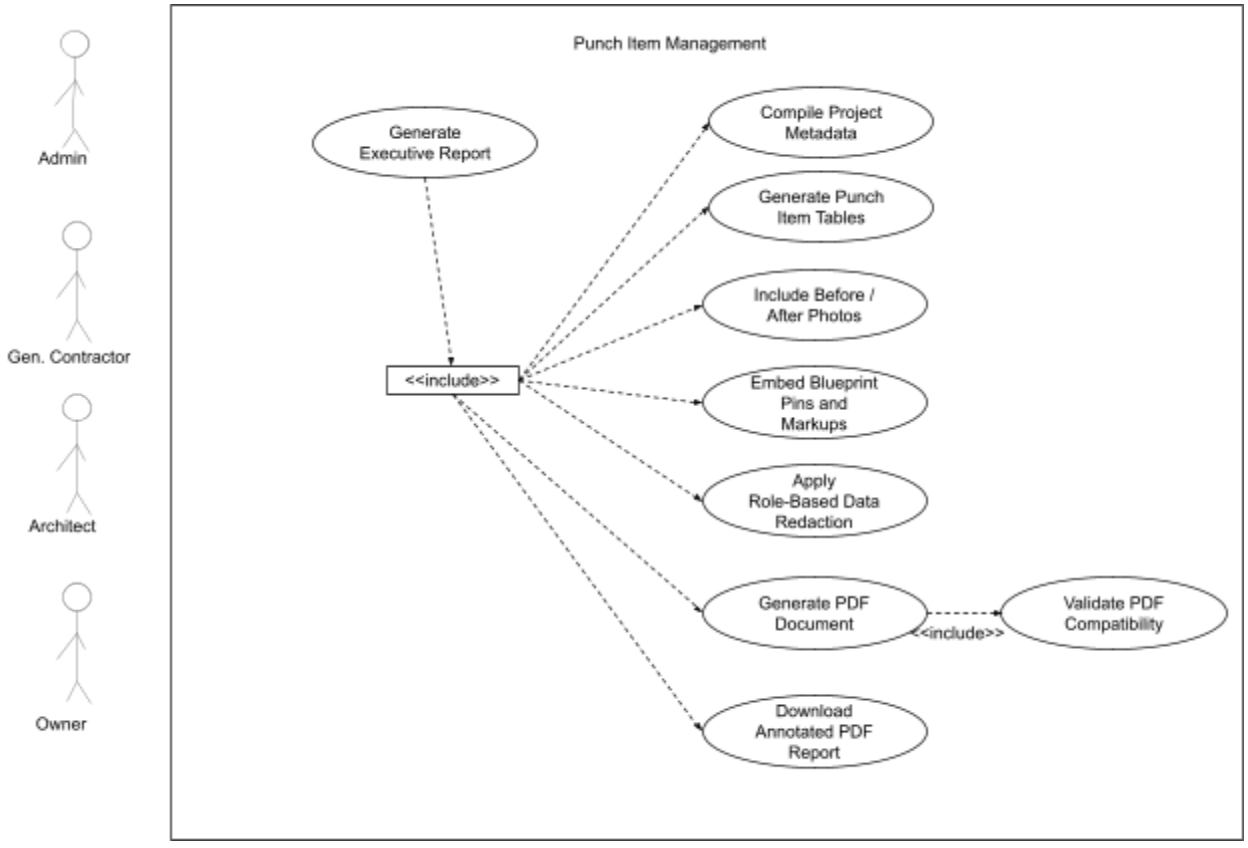
- FR-MKP-02: Available markup tools shall include revision cloud, rectangle, arrow, freehand pen, and dimension line.

- FR-MKP-03: The canvas engine shall maintain a local undo/redo vector stack during editing sessions.

- FR-MKP-04: Markups shall be serialized as vector coordinate JSON payloads linked to the specific sheet version and persisted to the server.

- FR-MKP-05: Saved markups shall be visible to all project members viewing that sheet version and shall be included in exported PDF drawings.

UC-10: Download an Annotated PDF Report



- FR-EXP-01: Users holding the Admin, General Contractor, Architect, or Owner role can trigger executive report generation.

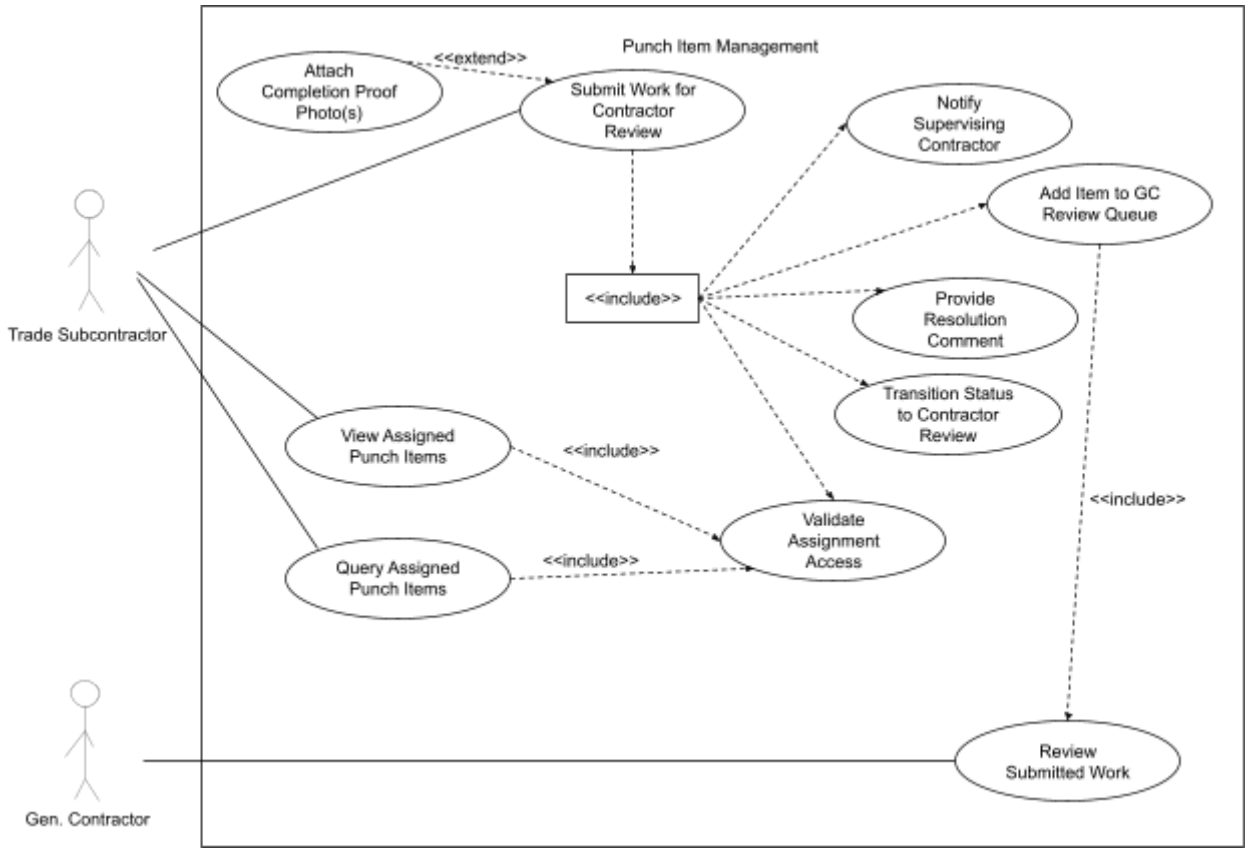
- FR-EXP-02: The reporting engine shall generate a formal closeout PDF document including project metadata, punch item tables, deficiency descriptions, assignees, dates, and before/after photos.

- FR-EXP-03: The reporting engine shall embed canvas vector pins and markups directly onto the original high-resolution blueprint plan sheets using standard vector drawing primitives.

- FR-EXP-04: PDF downloads shall be fully compatible with standard PDF reader applications (Adobe Acrobat, Preview, Chrome, Edge).

- FR-EXP-05: Exported reports shall redact any punch items or trade financial data that the requesting user's role is not authorized to inspect (see BR-02, BR-18).

UC-11: Subcontractor Updates Assigned Work



- FR-SUB-01: A Trade Subcontractor can view and query only punch items assigned directly to their company or their individual user account (see BR-02).

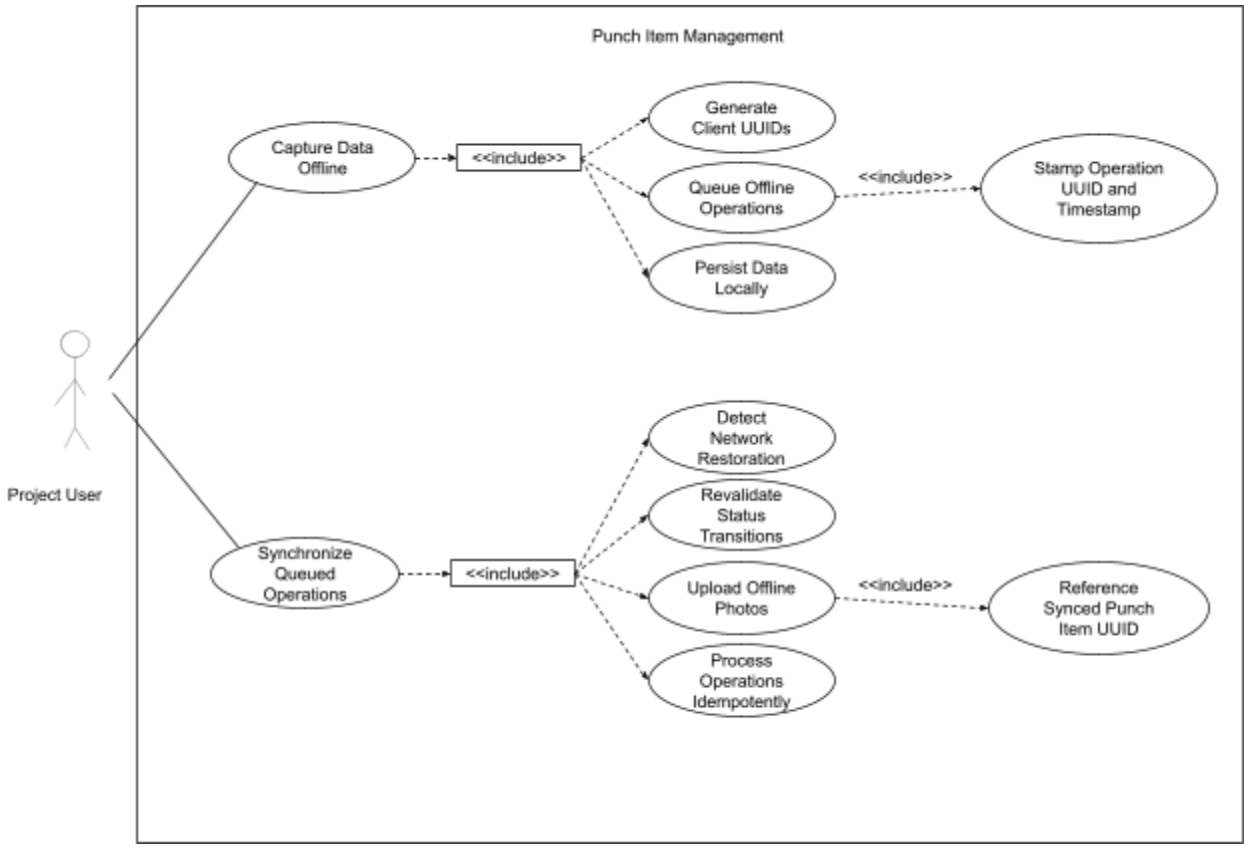
- FR-SUB-02: A Trade Subcontractor can transition an assigned item from In Progress to Contractor Review upon completing corrective work.

- FR-SUB-03: Advancing an item to Contractor Review shall require a non-empty resolution comment describing the remediation performed.

- FR-SUB-04: The system shall permit the subcontractor to attach one or more completion proof photos during the review submission.

- FR-SUB-05: Successful submission shall place the item into the General Contractor's review queue and emit a notification to the supervising contractor.

UC-12: Offline Data Capture & Sync



- FR-SYNC-01: When network connectivity is lost, the client shall persist new punch pins, status updates, comments, and photos locally using Dexie.js (IndexedDB).

- FR-SYNC-02: All entities created offline shall use client-generated UUID primary keys to ensure key validity upon server synchronization.

- FR-SYNC-03: Offline mutations shall be placed into an ordered local queue, each stamped with an operation UUID and client timestamp.

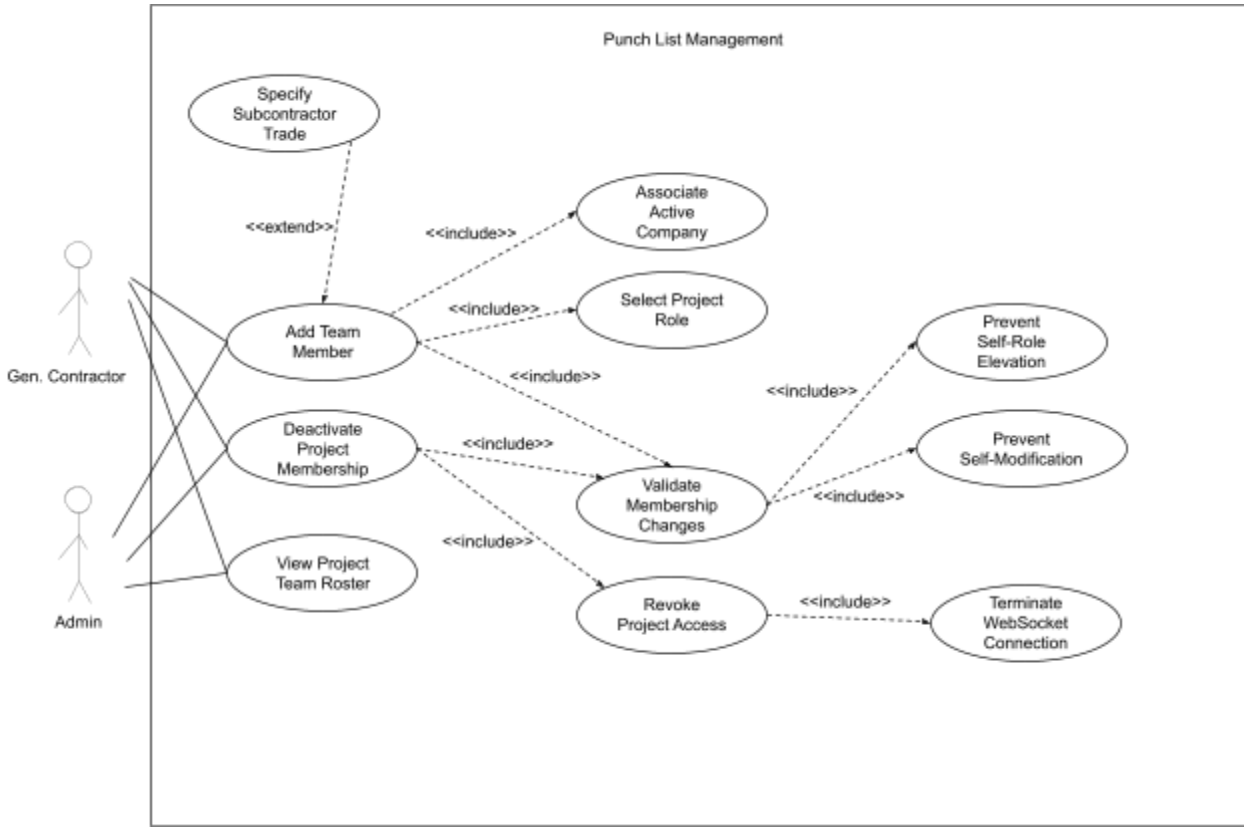
- FR-SYNC-04: When connectivity is restored, the client shall send queued operations to the bulk sync endpoint (POST /api/sync/reconcile) in chronological order.

- FR-SYNC-05: The server shall process operation UUIDs idempotently; previously applied operations shall be treated as no-ops without duplicating records.

- FR-SYNC-06: All status transitions submitted from the sync queue shall be re-evaluated against validate_status_transition at server sync time (see BR-07).

- FR-SYNC-07: Photos captured offline shall be uploaded via separate multipart requests referencing the already-synced punch item UUID.

UC-13: Manage Team Members



- FR-MEM-01: An Admin or General Contractor can view the project team roster and add users with specified project roles: General Contractor, Subcontractor, Architect, Owner, or Inspector.

- FR-MEM-02: Adding a user requires associating them with an active company record and, for subcontractors, a specific trade classification.

- FR-MEM-03: An Admin or General Contractor can deactivate a user's project membership, immediately revoking their access to project sheets and pins (see BR-04).

- FR-MEM-04: The system shall reject any attempt by a user to modify their own project membership record or elevate their own role (see BR-05).

- FR-MEM-05: Revoking a membership shall immediately terminate that user's active WebSocket connection to the project room.

6. Data Requirements

6.1 - User

- id (primary key, UUID/TEXT), email (unique, indexed), password_hash (bcrypt hash string), full_name, system_role (admin, contractor, subcontractor, architect, owner, field_inspector), company_id (foreign key to Company, nullable), phone, avatar_url, is_active (boolean, default true), created_at (timestamp), updated_at (timestamp), last_login_at (timestamp, nullable). Stores user credentials, global persona attributes, and company affiliation.

6.2 - Company

- id (primary key, UUID/TEXT), name (indexed), company_type (general_contractor, subcontractor, architecture, owner), trade (required when company_type is subcontractor; e.g., Electrical, Plumbing, HVAC, Framing), trade_initials (2-letter code; e.g., EL, PL, HV), phone, email, license_number, address, contact_person, is_active (boolean, default true), created_at (timestamp). Represents corporate entities engaged on job sites.

6.3 - ProjectCompany

- id (primary key, UUID/TEXT), project_id (foreign key to Project, indexed), company_id (foreign key to Company, indexed), role_on_project (prime, sub, design, owner), parent_company_id (foreign key to Company, nullable; references the General Contractor this subcontractor reports to on this specific project), is_active (boolean, default true), attached_at (timestamp). Establishes project attachment, trade scope, and multi-tier contractor-subcontractor reporting hierarchies (see FR-ORG-03, FR-ORG-04). Unique on (project_id, company_id).

6.4 - Project

- id (primary key, UUID/TEXT), name, project_number (unique, indexed), address, description, owner_company_id (foreign key to Company, nullable), gc_user_id (foreign key to User, nullable), status (active, archived, closed), start_date (date/text), target_completion (date/text), created_by (foreign key to User), created_at (timestamp), updated_at (timestamp), archived_at (timestamp, nullable). Master record for construction sites and punch walkthrough workspaces.

6.5 - ProjectMembership

- id (primary key, UUID/TEXT), project_id (foreign key to Project, indexed), user_id (foreign key to User, indexed), project_role (admin, contractor, subcontractor, architect, owner, field_inspector), invited_by (foreign key to User, nullable), joined_at (timestamp), is_active (boolean, default true).

Unique on (project_id, user_id). Binds an individual user to a project with a project-scoped security role (see FR-MEM-01).

6.6 - RefreshToken

- id (primary key, UUID/TEXT), user_id (foreign key to User, indexed), token_hash (cryptographic hash of token secret), issued_at (timestamp), expires_at (timestamp, 30-day lifespan), revoked_at (timestamp, nullable), device_label (text, e.g., "iPad Pro Field Unit 3"). Manages long-lived offline sessions and secure access token refresh cycles (see FR-AUTH-05).

6.7 - ChangeLog

- seq (primary key, BigInt auto-increment), project_id (foreign key to Project, indexed), entity_type (project, sheet, pin, comment, photo, markup, membership), entity_id (TEXT/UUID), operation (create, update, delete), payload (JSON text containing delta fields or tombstone), actor_user_id (foreign key to User), server_timestamp (timestamp with microsecond precision). Backs the offline delta sync engine, providing ordered replay cursors for client synchronization (see FR-SYNC-01, FR-SYNC-02).

6.8 - ClientOperation

- client_operation_id (primary key, UUID), user_id (foreign key to User, indexed), project_id (foreign key to Project), received_at (timestamp), resulting_entity_id (TEXT/UUID, nullable), result_status (applied, rejected, superseded), rejection_reason (text, nullable). Provides server-side idempotency tracking to prevent duplicate entity creation during offline queue sync retries (see FR-SYNC-05, FR-SYNC-06).

6.9 - Sheet

- id (primary key, UUID/TEXT), project_id (foreign key to Project, indexed), sheet_number (e.g., A-101, E-201), name (e.g., Level 2 Architectural Plan), discipline (architectural, structural, mechanical, electrical, plumbing, civil), floor_level (text), building_name (text, nullable), width_pts (integer, default 1200), height_pts (integer, default 800), file_path (text, remote storage URI), thumbnail_url (text, nullable), version_number (integer, default 1), scale_ratio (text, default 1/4" = 1'-0"), uploaded_by (foreign key to User), created_at (timestamp), updated_at (timestamp). Represents architectural drawing sets and vector PDF plan sheets.

6.10 - SheetVersion

- id (primary key, UUID/TEXT), sheet_id (foreign key to Sheet, indexed), project_id (foreign key to Project), version_number (integer, e.g., 1, 2, 3), version_tag (text, e.g., Rev 2 - Addendum 4), file_path (text), file_size_bytes (integer), change_summary (text), uploaded_by (foreign key to User), is_current (boolean, default true), created_at (timestamp). Preserves drawing addenda and historical blueprint revisions while maintaining coordinate continuity for pins (see FR-SHT-04, BR-14).

6.11 - Pin (Punch List Item)

- id (primary key, UUID/TEXT), project_id (foreign key to Project, indexed), sheet_id (foreign key to Sheet, indexed), item_number (sequential integer per project), title (text), description (text), status (open, in_progress, contractor_review, admin_review, closed), priority (low, medium, high, critical), discipline (text, default architectural), room_location (text), stamp_initials (2-letter trade code; e.g., EL, PL), x_pct (float, normalized percentage coordinate $[0.00, 100.00]$), y_pct (float, normalized percentage coordinate $[0.00, 100.00]$), due_date (date/text, nullable), created_by_user_id (foreign key to User), closed_at (timestamp, nullable), deleted_at (timestamp, nullable for soft deletes), created_at (timestamp), updated_at (timestamp). Master task entity spatially anchored to blueprints.

6.12 - PinAssignee

- id (primary key, UUID/TEXT), pin_id (foreign key to Pin, indexed), assignee_type (company or user), company_id (foreign key to Company, nullable), user_id (foreign key to User, nullable), assigned_by (foreign key to User), assigned_at (timestamp). Enforces the structural constraint of no more than two total assignee records per punch pin (maximum one company and one individual; see FR-ORG-05).

6.13 - PinPhoto

- id (primary key, UUID/TEXT), pin_id (foreign key to Pin, indexed), url (text, remote object storage URI), thumbnail_url (text, nullable), type (issue, progress, completion), caption (text, nullable), uploaded_by_id (foreign key to User), file_size_bytes (integer), created_at (timestamp). Stores photographic evidence for punch item identification and completion verification (see FR-MED-03, BR-11).

6.14 - Comment

- id (primary key, UUID/TEXT), pin_id (foreign key to Pin, indexed), author_id (foreign key to User), body (text), parent_id (foreign key to Comment, nullable for threaded replies), is_rejection_reason (boolean, default false), created_at (timestamp), updated_at (timestamp). Chronological discussion thread attached to individual punch deficiencies (see FR-COM-02).

6.15 - Markup

- id (primary key, UUID/TEXT), sheet_id (foreign key to Sheet, indexed), author_id (foreign key to User), type (cloud, arrow, rectangle, pen, measurement), data (JSON text defining vector coordinate points, control handles, and bounding box), stroke_color (hex string, default #DC2626), stroke_width (integer, default 4), fill_color (hex string, nullable), label (text, nullable), created_at (timestamp). Blueprint annotations drawn on the vector overlay canvas (see FR-MKP-02).

6.16 - AuditLog

id (primary key, UUID/TEXT), project_id (foreign key to Project, indexed), entity_type (text; e.g., pin, sheet, membership), entity_id (TEXT/UUID), actor_user_id (foreign key to User), actor_name (text), actor_role (text), action (CREATE, UPDATE, DELETE, STATUS_CHANGE), old_values (JSON text, nullable), new_values (JSON text, nullable), details (text), created_at (timestamp with microsecond precision). Immutable, append-only operational audit log for project compliance and dashboard activity feeds (see BR-16, FR-AUD-01).

7. External Interface Requirements

Method	Path	Auth	Purpose
POST	`/api/auth/login`	None (Public)	Authenticate user via email and password; returns JWT access token and refresh token (see FR-AUTH-03).
POST	`/api/auth/refresh`	None (Refresh Token)	Exchange a valid 30-day refresh token for a newly signed 60-minute JWT access token (see FR-AUTH-05).
POST	`/api/auth/logout`	Bearer JWT	Revoke the presented refresh token and invalidate client session (see FR-AUTH-08).
GET	`/api/auth/me`	Bearer JWT	Retrieve profile, system role, and company affiliation of currently authenticated user.
POST	`/api/users`	Bearer JWT (admin)	Provision a new user account with assigned system role and company affiliation; no open self-registration (see FR-AUTH-01).
GET	`/api/users`	Bearer JWT (admin)	List all provisioned users across the system for directory assignment.
GET	`/api/companies`	Bearer JWT	List all registered companies, trade classifications, and contact records.
POST	`/api/companies`	Bearer JWT (admin, contractor)	Register a new company entity (General Contractor, Subcontractor, Architect, Owner) with trade details (see FR-ORG-01).
PUT	`/api/companies/{company_id}`	Bearer JWT (admin, contractor)	Update company contact details, trade assignment, or license metadata.
DELETE	`/api/companies/{company_id}`	Bearer JWT (admin)	Safely deactivate a company record (is_active = false); rejects permanent deletion if referenced in pins or projects (see BR-09, FR-ORG-09).
POST	`/api/projects/{project_id}/companies`	Bearer JWT (admin, contractor)	Attach a company to a project, designating its operational role and

Method	Path	Auth	Purpose
			reporting contractor hierarchy (see FR-ORG-03, FR-ORG-04).
GET	`/api/projects`	Bearer JWT	List active projects for which the requesting user holds explicit membership; system admins can list all projects (see FR-MEM-06).
POST	`/api/projects`	Bearer JWT (project.create / admin)	Create a new project workspace with unique project code, name, address, and target closeout date (see FR-PRJ-01, FR-PRJ-02).
GET	`/api/projects/{project_id}`	Bearer JWT (Project Member)	Retrieve project workspace details and configuration metadata. Returns 404 if user lacks membership (see FR-MEM-07).
PUT	`/api/projects/{project_id}`	Bearer JWT (project.update / admin, contractor)	Update project metadata, physical site address, and schedule dates (see FR-PRJ-03).
POST	`/api/projects/{project_id}/archive`	Bearer JWT (admin)	Transition project status to archived, placing project, sheets, and pins into a strict read-only lock (see BR-08, FR-PRJ-05).
GET	`/api/projects/{project_id}/summary`	Bearer JWT (Project Member)	Return KPI summary metrics: counts of punch items aggregated by status, priority, and trade (see FR-PRJ-08).
GET	`/api/projects/{project_id}/capabilities`	Bearer JWT (Project Member)	Return the effective capability set of the requesting principal for UI permission gating (see FR-RBAC-07).
GET	`/api/projects/{project_id}/members`	Bearer JWT (Project Member)	List all active project team members, project roles, and affiliated trades.
POST	`/api/projects/{project_id}/members`	Bearer JWT (admin, contractor)	Add a provisioned user to a project with a project-scoped security role (see FR-MEM-04).
PUT	`/api/projects/{project_id}/members/{user_id}`	Bearer JWT (admin, contractor)	Modify a member's project role; self-modification or self-elevation is blocked (see BR-05, FR-MEM-05).

Method	Path	Auth	Purpose
DELETE	`/api/projects/{project_id}/members/{user_id}`	Bearer JWT (admin, contractor)	Deactivate a user's project membership, instantly revoking project access and terminating socket sessions (see FR-MEM-04).
GET	`/api/projects/{project_id}/sheets`	Bearer JWT (Project Member)	List all architectural and engineering drawing sheets uploaded to the project.
POST	`/api/projects/{project_id}/sheets`	Bearer JWT (sheet.manage / admin, contractor, architect)	Upload a new drawing sheet (multipart/form-data: PDF/PNG/JPG) with sheet number, title, and discipline tag (see FR-SHT-01).
GET	`/api/projects/{project_id}/sheets/{sheet_id}`	Bearer JWT (Project Member)	Retrieve sheet details, viewport dimensions, and historical revision log.
PUT	`/api/projects/{project_id}/sheets/{sheet_id}`	Bearer JWT (sheet.manage)	Update sheet metadata (title, floor level, discipline classification, or scale ratio).
DELETE	`/api/projects/{project_id}/sheets/{sheet_id}`	Bearer JWT (admin)	Soft-delete a drawing sheet; rejects deletion if unresolved punch pins are attached (see FR-SHT-06).
POST	`/api/projects/{project_id}/sheets/{sheet_id}/versions`	Bearer JWT (sheet.manage)	Upload a new drawing sheet revision; automatically propagates existing pins and markups to new version (see BR-14, FR-SHT-04).
GET	`/api/projects/{project_id}/pins`	Bearer JWT (Project Member)	List punch pins filtered by sheet, status, priority, or trade. Subcontractors receive only their assigned items (see BR-02, FR-ORG-08).
POST	`/api/projects/{project_id}/pins`	Bearer JWT (pin.create / admin, contractor, architect, inspector)	Drop a new punch pin at normalized percentage coordinates (x_pct, y_pct) and broadcast via WebSocket (see FR-PIN-01, FR-PIN-02).

Method	Path	Auth	Purpose
GET	`/api/projects/{project_id}/pins/{pin_id}`	Bearer JWT (Project Member)	Retrieve full punch item details, spatial coordinates, assignee roster, photos, comments, and audit log.
PUT	`/api/projects/{project_id}/pins/{pin_id}`	Bearer JWT (Authorized Pin Editor)	Update punch item details (title, room location, trade classification, due date, description).
PATCH	`/api/projects/{project_id}/pins/{pin_id}/status`	Bearer JWT (Role-Validated)	Transition item status. Enforces 5-stage lifecycle state machine; rejects unauthorized closure attempts (see BR-01, BR-07, FR-STAT-02).
PUT	`/api/projects/{project_id}/pins/{pin_id}/assignees`	Bearer JWT (admin, contractor)	Assign punch item to a subcontractor company, individual user, or both (capped at 2 entries max; see FR-ORG-05, FR-ASN-03).
DELETE	`/api/projects/{project_id}/pins/{pin_id}`	Bearer JWT (pin.delete / admin)	Soft-delete a punch pin (deleted_at timestamp). Blocked for non-admin roles (see BR-01, FR-PIN-07).
GET	`/api/projects/{project_id}/pins/{pin_id}/photos`	Bearer JWT (Project Member)	List all issue and completion proof photos attached to a punch item.
POST	`/api/projects/{project_id}/pins/{pin_id}/photos`	Bearer JWT (Authorized Pin Editor)	Upload a photo attachment (multipart image file) linked to the punch pin with type issue or completion (see FR-MED-03).
DELETE	`/api/projects/{project_id}/pins/{pin_id}/photos/{photo_id}`	Bearer JWT (admin, or Uploader)	Delete an attached photo record and remove corresponding file from object storage.
GET	`/api/projects/{project_id}/pins/{pin_id}/comments`	Bearer JWT (Project Member)	Retrieve chronological comment thread for a specific punch item.
POST	`/api/projects/{project_id}/p`	Bearer JWT (Project Member)	Append a comment to the punch item thread with author role and timestamp (see FR-COM-01, FR-COM-02).

Method	Path	Auth	Purpose
	`ins/{pin_id}/comments`		
GET	`/api/sheets/{sheet_id}/markups`	Bearer JWT (Project Member)	Fetch all vector markups (clouds, arrows, rectangles, measurements) for a specific sheet.
POST	`/api/sheets/{sheet_id}/markups`	Bearer JWT (admin, contractor, architect)	Save vector markup annotations drawn on the blueprint overlay canvas (see FR-MKP-04).
DELETE	`/api/sheets/{sheet_id}/markups/{markup_id}`	Bearer JWT (admin, or Markup Author)	Delete a vector markup annotation from a sheet.
GET	`/api/sync/hydrate/{project_id}`	Bearer JWT (Project Member)	Hydrate client-side Dexie.js (IndexedDB) with current working set of sheets, pins, comments, and members (see FR-SYNC-12).
GET	`/api/sync/delta/{project_id}`	Bearer JWT (Project Member)	Fetch incremental change-log records past a client sequence cursor; filtered by user role scope (see FR-SYNC-02, FR-SYNC-03).
POST	`/api/sync/reconcile`	Bearer JWT (Project Member)	Bulk upload ordered offline mutations; verifies operation UUIDs idempotently and re-validates state transitions (see FR-SYNC-05, FR-SYNC-08).
GET	`/api/projects/{project_id}/reports/pdf`	Bearer JWT (admin, contractor, architect, owner)	Generate and stream executive PDF punch list report with embedded before/after photos and sign-off blocks (see FR-EXP-01).
GET	`/api/projects/{project_id}/reports/annotated-plan`	Bearer JWT (admin, contractor, architect, owner)	Generate composite architectural PDF with baked-in vector pins, trade stamps, and markup clouds (see FR-EXP-02).
GET	`/api/projects/{project_id}/reports/csv`	Bearer JWT (admin, contractor, architect, owner)	Export structured tabular CSV punch list data for external ERP/BIM system import (see FR-EXP-04).
GET	`/api/audit-logs`	Bearer JWT (admin, contractor, owner)	Query immutable audit log stream filtered by project_id, pin_id, or date range (see FR-AUD-04).

Method	Path	Auth	Purpose
GET	`/api/health`	None (Public)	Health check endpoint returning server status, database connectivity, and engine version (see FR-AUTH-07).

8. Nonfunctional Requirements

8.1 - Security

- Passwords are hashed with `bcrypt` at a work factor (salt rounds) of at least 12; plaintext passwords are never written to the database, application logs, or API response payloads (see BR-10).
- The JWT signing secret and database credentials are provided strictly via environment variables (`.env`) and are never committed to version control.
- All database interactions use parameterized queries through SQLAlchemy 2.0 ORM; raw user input is never concatenated into SQL statements, preventing SQL injection vulnerabilities.
- Protected endpoints strictly validate the cryptographic signature and expiration of JWT bearer tokens on every request; expired or tampered tokens immediately return HTTP 401 Unauthorized.
- Failed authentication attempts return identical, generic error messages (`"Invalid credentials"`) and HTTP 401 status codes to eliminate user enumeration vectors.
- Subcontractor data isolation is enforced at the database query layer: all pin, comment, and export queries automatically inject tenant-scoping clauses (`company_id == user.company_id`) so unauthorized trade records are never transmitted over the network (see BR-02, FR-ORG-08).

8.2 - Performance

- Server-side capability and project membership checks add no more than 50 ms to the 95th-percentile (p95) API request latency.

- Project and membership lookups executed on authenticated requests are served directly from composite database indexes on `(project_id, user_id)` and `(project_id, status)`.
- An incremental delta sync response covering up to 500 changed records past a client cursor returns within 2.0 seconds on standard development and production environments (see FR-SYNC-02).
- The dual-layer blueprint rendering engine maintains a smooth 60 FPS frame rate during continuous panning and trackpad/pinch zooming on tablet viewports up to 800% zoom.
- Vector pin stamps, trade indicators, and markup layers render on the blueprint canvas within 500 ms of sheet loading or WebSocket broadcast receipt.
- Real-time WebSocket event broadcasts (`pin:created`, `pin:updated`, `pin:deleted`) over Socket.IO deliver to connected project clients within 500 ms of backend transaction commit.

8.3 - Reliability

- Each operation within an offline sync batch is executed inside an isolated database savepoint/transaction, ensuring that a single invalid operation cannot corrupt or partially apply other valid mutations in the batch (see FR-SYNC-09).
- The tablet client remains fully operational in read and queued-write mode for at least 8 hours of disconnected field walkthroughs, persisting blueprints, pins, and photos locally in IndexedDB via Dexie.js (see FR-SYNC-01, FR-SYNC-02).
- Spatial pin coordinates are stored and calculated strictly as normalized floating-point percentages (x_{pct}, y_{pct}), guaranteeing zero coordinate drift or placement distortion regardless of device aspect ratio, display pixel density, or zoom factor (see BR-12, FR-PIN-02).
- Re-established network connections automatically trigger chronological queue synchronization with idempotent client operation UUIDs, guaranteeing zero duplicate records or lost field data (see FR-SYNC-05, FR-SYNC-06).

8.4 - Maintainability

- The role-to-capability map and resource-scoping policies are defined as declarative data structures within a single policy module (`backend/app/permissions.py`) rather than procedural branching logic scattered across endpoint handlers, allowing the access model to be reviewed and unit-tested in isolation (see FR-RBAC-02).
- Punch item status transitions are governed by a centralized, unit-tested finite state machine service (`validate_status_transition`), ensuring lifecycle integrity across both REST API calls and offline batch sync ingestion (see BR-07).
- The frontend architecture enforces clean component styling via centralized CSS custom properties (design tokens in `theme.css`), prohibiting hardcoded color hex values and ad-hoc padding constants in component files.
- The backend relies on standard ASGI FastAPI patterns and the pure-Python `pg8000` driver, avoiding native C-compiler extensions to ensure seamless portability across Windows, Linux, and macOS development environments.

8.5 - Usability & Tablet Ergonomics

- All interactive touch targets (pin buttons, toolbar controls, status toggles, drawer handles) maintain a minimum hit area of 38px × 38px to accommodate one-handed and gloved tablet use on active job sites.
- The user interface is fully responsive and optimized for portrait and landscape orientations on 768px-wide tablets (e.g., standard 10.2" iPad and Android field tablets).
- Critical safety blocker pins feature a high-contrast pulsating halo shader to ensure instant visual identification in high-ambient-light outdoor field environments (see BR-19).
- Core inspection flows (dropping a pin, capturing photo evidence, updating status) require no more than three taps from the primary blueprint viewport.

8.6 - Portability & Interoperability

- Generated PDF closeout reports and annotated plan sets adhere strictly to standard PDF specifications (PDF 1.7 / ISO 32000-1) and can be opened, viewed, and printed by any standard reader (Adobe Acrobat, Apple Preview, Google Chrome) without requiring third-party plugins (see BR-16, FR-EXP-03).
- Tabular CSV exports follow RFC 4180 specifications with UTF-8 encoding, enabling seamless direct ingestion into industry-standard construction ERP platforms (Autodesk BIM 360, Procore).
- The database abstraction layer supports automatic environment detection, running seamlessly against Cloud PostgreSQL in production and zero-configuration SQLite for local offline development.

8.7 - Auditability

- All state-altering mutations (pin drops, status transitions, reassignments, sheet revision uploads, and membership changes) write an immutable record to the `audit_logs` table containing actor ID, entity reference, old/new value snapshots, and microsecond UTC timestamps (see BR-16, FR-AUD-01).
- Audit log records are append-only; the database schema and application service layer prohibit any `UPDATE` or `DELETE` operations on historical audit rows, even by system administrators.

9. Reporting Requirements and Supplemental Information

9.1 - Report 1

- 9.1.1: The system must support one-click generation of:
    1. An Annotated Plan Set PDF - the original blueprint sheets with punch pins and vector markups embedded directly on the drawings.
    2. A Punch Report PDF -a formal document listing all punch items with their descriptions, status history, assigned user, location reference, and embedded photos.
    3. A CSV Export -a spreadsheet of all punch items and their metadata for use in external project management tools.
- 9.1.2: Reports must be scoped to the active project and reflect the current state of all pins at the time of export.
- 9.1.3: All generated files must be openable by any standard PDF or spreadsheet application without proprietary software.
- 9.1.4: The dashboard must display real-time KPI cards: Resolution Rate (%), Active/In Progress count, Items in Review count, and Critical Blockers count.

10. Glossary of Terms

- 10.1: Punch List - A construction industry document listing outstanding defects, incomplete work, or QA/QC items that must be resolved before project closeout
- 10.2: Punch Walkthrough - The on-site inspection performed just prior to project completion where the project team walks the building and documents all outstanding issues
- 10.3: Punch Item / Pin - A digital marker dropped on a blueprint at a specific coordinate to represent a punch list task
- 10.4: Markup - A vector annotation (revision cloud, arrow, rectangle, dimension line) drawn on a blueprint to spatially highlight an area of concern
- 10.5: Blueprint Viewer - The interactive PDF.js + Canvas rendering engine used to display and annotate construction plans
- 10.6: Status Lifecycle - The enforced sequence of states a punch item progresses through: Open -> In Progress -> Contractor Review -> Admin Review -> Closed

- 10.7: RBAC - Role-Based Access Control — permissions are assigned by role, not by individual user
- 10.8: Offline Sync Queue - A Dexie.js-managed list of mutations captured while offline, replayed to the server on reconnection
- 10.9: Revision Cloud - A CAD-standard annotation shape (scalloped cloud outline) used to highlight areas of change on a blueprint
- 10.10: JWT - JSON Web Token — a signed token issued at login used to authenticate all subsequent API requests
- 10.11: Audit Log - A server-side record of every significant data change, capturing actor, timestamp, and field-level delta
- 10.12: PlanGrid Autodesk PlanGrid — the reference enterprise platform for construction field documentation; the primary inspiration for this software

11. Appendix

- 11.1: Frontend Framework - React 18 + Vite (ES6+, JSX)
- 11.2: Styling - Vanilla CSS3, CSS Custom Properties, Glassmorphic design
- 11.3: Blueprint Rendering - PDF.js + HTML5 Canvas API
- 11.4: Offline Storage - Dexie.js (IndexedDB wrapper)
- 11.5: Backend - Python 3.14, FastAPI, Uvicorn
- 11.6: Database - Supabase (PostgreSQL) primary / SQLite fallback via SQLAlchemy 2.0
- 11.7: Real-Time - Socket.IO (python-socketio AsyncServer)
- 11.8: Auth - JWT (python-jose) + bcrypt (passlib)
- 11.9: Reporting - ReportLab + PyPDF

12. Approvals and Sign-off

By signing this document, the customer and developers agree to the details specified in this document, specifically: the scope of the project, the functional requirements, and the non-functional requirements

Name	Position	Date Reviewed	Signature	Date